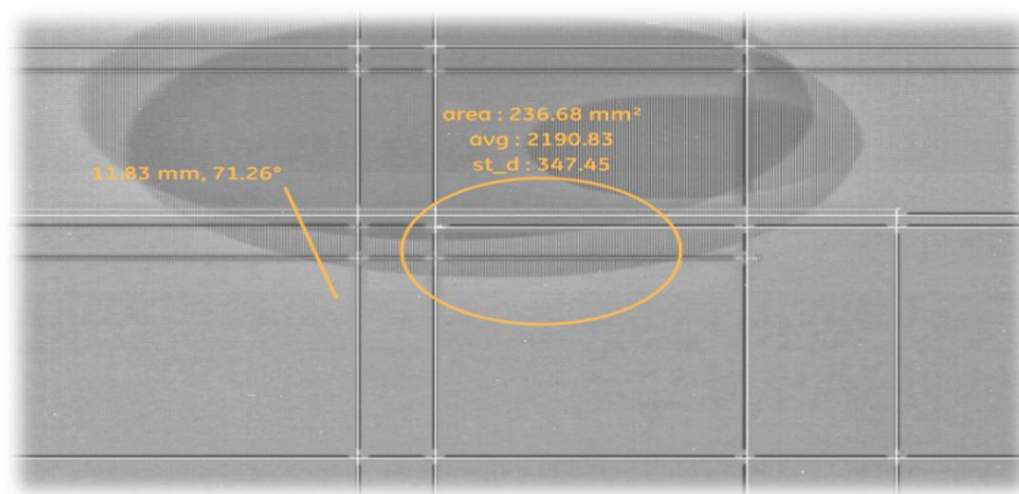


Rapport de stage de 4^{ème} année

8 Avril – 2 Août 2019

**Développement d'outils de mesure
en mammographie numérique**

*Maître de stage :***SERVANT Nicolas**

Lead Engineer

General Electric Healthcare

*Enseignant référent :***HAFIANE Adel**Enseignant chercheur à l'INSA
Centre-Val-De-Loire**GE Healthcare**

General Electric Healthcare, Buc (78)

Remerciements

Tout d'abord, je voudrais remercier M. Julien LEGER, manager de l'équipe Software Apps mammography, qui m'a permis d'effectuer ce stage au sein de son équipe. Je remercie également mon maître de stage, M. Nicolas SERVANT, Lead Engineer chez General Electric Healthcare, qui m'a incluse au sein de son équipe, m'a fait participer aux réunions quotidiennes.

Je souhaite ensuite adresser mes remerciements au corps professoral et administratif de l'INSA Centre-Val-De-Loire, pour son aide, ainsi qu'à M. Adel HAFIANE, enseignant référent, qui a suivi mon avancement pendant ce stage.

Je remercie chaleureusement Mme Lucie BOURSIER, Lead Engineer, et M. Bruno LE CORGNE, architecte logiciel, d'avoir répondu à toutes mes questions, de m'avoir conseillé au quotidien, ainsi que pour leur relecture attentive de mon travail. Je remercie également Benjamin DUVERLIE, ingénieur spécialiste qualité, qui m'a aidée à comprendre et implémenter des nouveaux tests automatiques dans mon code, ainsi que Alexis PONTIN, spécialiste en ingénierie logicielle, qui m'a donné de précieux conseils pour la partie graphique.

Je voudrais enfin exprimer ma reconnaissance envers tous mes collègues de l'équipe Software Apps Mammography, pour leur accueil, leur bonne humeur et leur bienveillance.

Résumé

La mammographie numérique est de plus en plus utilisée de nos jours, afin de détecter au plus vite les cancers du sein. General Electric (GE) Healthcare développe des mammographes et leurs logiciels associés. Ceux-ci proposent des outils pour effectuer et afficher des images de mammographie, et en particulier mettre en évidence des zones d'intérêt, en traçant des segments et des ellipses sur l'image.

Les utilisateurs du logiciel souhaitent, en plus de l'aire déjà affichée, avoir des informations statistiques sur la valeur des pixels inclus dans l'ellipse : la valeur minimale, maximale, la moyenne et l'écart-type. C'est dans ce contexte que se déroule ce stage : mettre au point une méthode de récupération de pixels inclus dans une ellipse, puis calculer et afficher les valeurs statistiques.

Ce projet s'inscrit dans le cadre du stage de quatrième année du cursus du département Sécurité et Technologies Informatiques de l'INSA Centre-Val-De-Loire, campus de Bourges. Il se déroule au sein de l'équipe Software Apps Mammography de GE Healthcare.

Afin d'atteindre ces objectifs, ce stage s'est divisé en trois phases : une partie mathématique théorique, une partie pratique, avec l'application des théories mathématiques dans le logiciel, et enfin une partie de tests et optimisation du code. Les objectifs ont été atteints : les valeurs statistiques sont affichées lors de la création d'une ellipse, et mis à jour lors de modifications (taille ou emplacement dans l'image).

Cette fonctionnalité sera utilisée dans les futures versions du logiciel.

Mots clés : Développement algorithmique, Optimisation, Programmation en C++ et QML, Tests unitaires

Abstract

Digital mammography is increasingly being used nowadays to detect breast cancer in its earlier stages. General Electric (GE) Healthcare develops mammography machines and software that provides tools to display mammography images, and in particularly highlighting areas of interest in the image.

In addition to the area being already displayed, users of the software would also like to have statistical information on the value of the pixels included in the ellipse such as: the minimum, maximum, average and standard deviation.

This internship takes place in the context of developing a method for obtaining pixels included in an ellipse, then calculating and displaying statistical values.

This internship is part of the 4th year internship for the Security and IT Technology department of the INSA Centre-Val-De-Loire, Bourges campus. It takes place within the GE Healthcare Software Apps team.

In order to achieve the required objectives, this internship was divided into three phases: a theoretical mathematical part, a practical part, with the application of mathematical theories in the software, and finally a testing and code optimization phase. The previous stated objectives have been achieved: The statistical values are displayed when an ellipse is created and updated when changes are made (size or location in the image).

This valuable feature will be used and implemented in all future versions of the software.

Keywords : Algorithmic development, Optimization, Programming in C++ and QML, Unit tests

Table des matières

Remerciements	1
Résumé	2
Abstract	2
Introduction	4
1. Contexte	5
1.1 Historique de General Electric	5
1.2 General Electric Healthcare	6
1.3 Contexte du stage	7
1.3.1 Présentation du service	7
1.3.3 Présentation du mammographe et du logiciel	9
2. Objectifs du stage	12
3. Travail réalisé	13
3.1 Travail préliminaire	13
3.2 Premiers pas avec le logiciel	15
3.3 Calculs statistiques lors de la création et modification d'une ellipse	17
3.3.1 Création d'un objet graphique	17
3.3.2 Modification d'un objet graphique	18
3.4 Optimisation	19
3.5 Tests unitaires	21
3.6 Tests automatiques SATI	22
3.7 Corrections	23
4. Améliorations	23
4.1 Rotation de l'ellipse	23
4.1.1 Objectifs	23
4.1.2 Démarches	24
4.1.2.1 Obtention des pixels dans l'ellipse	24
4.1.2.2 Interface graphique	25
4.1.3 Limites	25
4.2 Histogrammes	26
Conclusion	28
Sources	29
Table des illustrations	30
ANNEXES	32

Introduction

L'histoire de l'imagerie médicale commence en 1895 avec les premiers travaux sur les rayons X. De nombreuses innovations techniques ont été réalisées par des chercheurs du monde entier. C'est au milieu des années 1950 que Jacob Gershon-Cohen, radiologue américain, réalise les premières mammographies numériques, afin de détecter au plus tôt les anomalies des tissus mammaires.

Le cancer du sein est le cancer le plus fréquent chez la femme : en 2017, près de 60000 nouveaux cas ont été diagnostiqués en France. Si cette maladie est encore responsable de plus de 10000 décès cette même année, le taux de mortalité a diminué de -1,5% par an en moyenne entre 2005 et 2012. Cette amélioration s'explique par un meilleur dépistage mais également par le développement de thérapies toujours plus efficaces.

General Electric Healthcare, filiale de la multinationale General Electric, est un acteur majeur de la production de mammographes et logiciels de mammographie. C'est au sein de l'équipe Software Apps Mammography de GE Healthcare à Buc (78) que s'effectue ce stage.

Afin d'aider à la revue des images de mammographie, l'utilisateur (radiologue ou physicien) a besoin d'outils de mesure, afin d'obtenir plus d'informations sur une zone d'intérêt de l'image. Actuellement, sur le système, il peut dessiner des segments et des ellipses, afin de connaître leur longueur ou leur aire.

Après analyse des retours des utilisateurs, un nouveau besoin est apparu : les utilisateurs souhaiteraient, en plus de l'aire, connaître des données statistiques relatives aux valeurs des pixels inclus dans l'ellipse (e.g. valeur minimale, maximale, moyenne, écart-type).

Le but de ce stage est de développer un algorithme afin d'effectuer des calculs sur les pixels de l'image inclus dans une ellipse, puis de développer l'interface utilisateur pour présenter les résultats des calculs.

Dans un premier temps, nous présenterons le contexte du stage, en présentant l'entreprise, l'équipe au sein de laquelle se déroule le stage, puis la technologie utilisée. Puis nous décrirons plus en détail les objectifs de ce stage, pour enfin montrer la démarche adoptée, les résultats obtenus, puis les améliorations possibles.

1. Contexte

1.1 Historique de General Electric

Historique

L'entreprise General Electric (GE) voit le jour en 1889 grâce à Thomas Edison, l'inventeur de la lampe à incandescence. D'abord connue sous le nom Edison General Electric Company, elle fusionne avec la Thomson-Houston Company en 1892 pour fonder la General Electric Company.

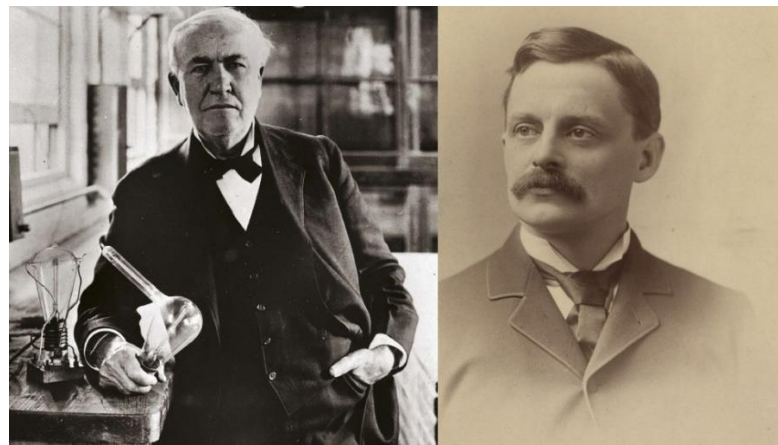


Figure 1 – Thomas Edison (vers 1920) et Elihu Thomson (vers 1880)

Aujourd'hui, GE est un des leaders mondiaux de l'imagerie et des équipements médicaux, ainsi que dans les domaines de l'énergie et du transport. L'entreprise fournit des équipements pour la production, le transport et la distribution d'électricité. Elle fournit également des réacteurs d'avion et des moteurs de locomotives, ainsi que des solutions de financement pour les particuliers et entreprises.

Secteurs d'activité

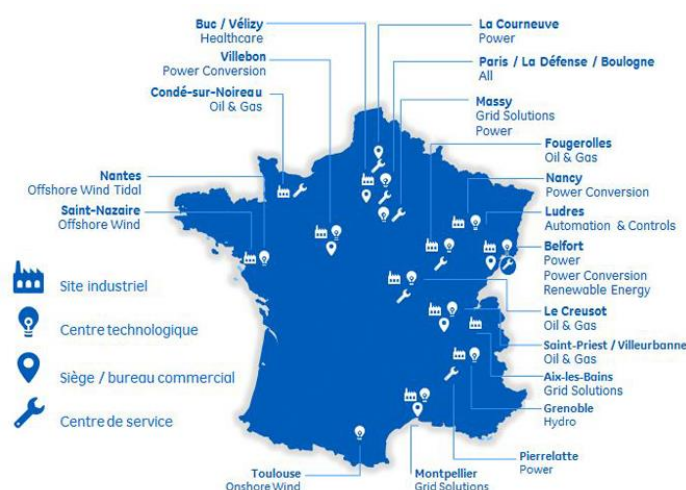


Figure 2 – Secteurs d'activités de General Electric en France

GE se compose de cinq branches principales d'activité :

- **GE Commercial Finance – GE Capital** : services financiers du groupe pour les particuliers et entreprises
- **GE Healthcare** : branche médicale
- **GE Industrial** : produits industriels et services (éclairage, systèmes de sécurité, appareils électroménagers ...)
- **GE Infrastructure** : réunion de trois branches
 - GE Power : énergies renouvelables, électricité
 - GE Oil & Gas : équipements et services pour l'industrie du pétrole et gaz naturel
 - GE Energy Service : services et conseils pour les entreprises liées à l'énergie
- **NBC Universal** : diffusion de chaînes de TV

Aujourd'hui, son chiffre d'affaires s'élève à 125 milliards de dollars et compte plus de 260000 employés à travers 150 pays (chiffres de 2018). Son siège social se trouve à Boston (Massachusetts). Son PDG actuel est Henry Lawrence Culp Jr, qui a succédé à Jeffery Immelt en 2017.

En 2016, General Electric se hisse à la huitième position dans le classement des vingt plus grosses capitalisations boursières.

GE en France

Avec ses 10000 salariés et son chiffre d'affaires de près de 7 milliards d'euros, GE est un acteur économique majeur dans l'Hexagone. En France, GE compte 2 sièges mondiaux et 3 sièges européens. Ses principaux sites de productions sont situés à Belfort (GE Power and Water), à Buc (GE Healthcare) et au Creusot (GE Oil & Gas). Engagé dans l'économie française, GE a noué des partenariats avec des centres de recherche, PME et grands groupes français tels que EDF, GDF Suez, Total ou Air France.

1.2 General Electric Healthcare

GE Healthcare est l'un des leaders mondiaux de la fabrication d'équipements d'imagerie médicale. Son siège social est situé à Chalfont St. Giles au Royaume-Uni. GE Healthcare emploie près de 50000 personnes dans plus de 100 pays. Grâce à son expertise dans de nombreux domaines de l'imagerie médicale et des technologies de l'information, GE Healthcare offre de nombreux services aux cliniciens afin de permettre des diagnostics plus précoces et plus fiables.

Les produits proposés permettent un meilleur traitement du cancer, des maladies cardiaques, des maladies neurologiques et autres pathologies. Ses concurrents principaux sont les groupes internationaux Siemens, Philips et Hologic aux Etats-Unis. GE Healthcare dépense chaque année plus d'un milliard de dollars dans la Recherche et le Développement. Les principaux sites dans le monde sont situés à Milwaukee (USA), Buc (France) et Tokyo (Japon).

GE Healthcare est divisé en différents domaines d'activités :

- **Diagnostic Imaging** : offre aux médecins des moyens rapides pour visualiser des fractures, diagnostiquer des traumatismes, ou encore identifier les stades précoces de cancers.
- **Global Services** : assure la maintenance d'une vaste gamme de systèmes et d'appareils médicaux dans le monde entier.

- **Clinical Systems** : offre aux cliniciens et aux administrateurs hospitaliers des services permettant d'améliorer la qualité et l'efficacité des soins prodigués.
- **Life Science** : offre des innovations dans les domaines de la recherche de nouveaux médicaments, de la fabrication des produits biopharmaceutiques ou encore des technologies cellulaires.
- **Medical Diagnostics** : développe des produits pharmaceutiques d'imagerie médicale utilisés en radiographie, IRM et médecine nucléaire.
- **Integrated IT** : propose des solutions IT cliniques et financières permettant aux clients de réduire leurs coûts et d'améliorer la qualité des soins.
- **Interventional, Cardiology and Surgery** : offre des outils et des technologies pour les soins interventionnels, cardiaques et chirurgicaux.

Avec près de 2000 salariés dont plus de 400 ingénieurs en Recherche et Développement, Buc est un site d'excellence mondial. Ce site réunit les experts des systèmes vasculaires, de la mammographie et des logiciels d'applications cliniques.

1.3 Contexte du stage

1.3.1 Présentation du service

Equipe Software Apps Mammography

Ce stage se déroule au sein de l'équipe Software Apps Mammography, managé par Mr Julien LEGER. L'équipe compte environ 25 ingénieurs. L'équipe s'occupe principalement du développement du logiciel de la station d'acquisition du mammographe Pristina (cf. figure 5).

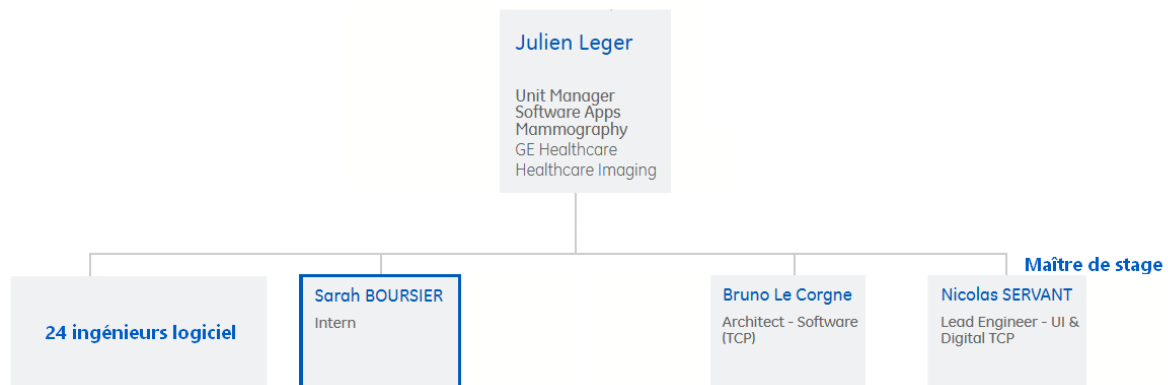


Figure 3 - Organigramme simplifié du service Software Apps Mammography de General Electric Healthcare (Buc)

Fonctionnement interne : la méthode Agile Scrum

Les équipes d'ingénierie logiciel du site de Buc ont fait le choix de la méthode Agile Scrum pour le développement de leurs produits. Cette méthode de gestion de projet permet d'améliorer l'efficacité et la productivité de l'équipe. Dans ce mode de fonctionnement, chacun a un rôle :

- **Scrum Master** : Facilite la communication au sein de l'équipe, il s'assure de la bonne avancée du projet. Il est en communication avec le Product Owner afin de déterminer l'ordre d'importance des fonctionnalités à implémenter.
- **Product Owner** : Il définit les spécifications fonctionnelles du logiciel, et valide les fonctionnalités développées. Il joue le rôle du client, afin de comprendre au mieux les attentes de ce dernier.
- **L'équipe de développement** : groupe de taille variable, de 10 à 200 personnes, selon les entreprises. L'équipe Software Apps Mammography est divisée en quatre équipes de développement.

Cette méthode permet la production en continu d'un logiciel, par multiples itérations. Le logiciel n'est pas conçu en une fois : il est découpé en un « backlog », qui regroupe toutes les fonctionnalités du logiciel. Celles-ci sont découpées en sous-fonctionnalités, implémentables rapidement. Leur définition constitue ce que l'on appelle « user story ». Celles-ci sont classées par priorité.

L'emploi du temps est découpé en « sprints », qui durent en général deux semaines. Chaque jour, le « stand up », une réunion d'une quinzaine de minutes avec l'équipe, permet de faire le point sur ce qui a été fait la veille, et ce qui est prévu de faire dans la journée. A la fin de chaque sprint, une nouvelle version du logiciel est livrée, et une réunion est prévue avec l'équipe afin de mettre en évidence les points positifs et négatifs, ainsi que des actions à effectuer pour s'améliorer au sprint suivant. Cela permet un développement efficace, et une bonne communication au sein du groupe.

Software Apps Mammography comporte quatre équipes Agile Scrum différentes. Mon stage se déroule au sein d'une des équipes.

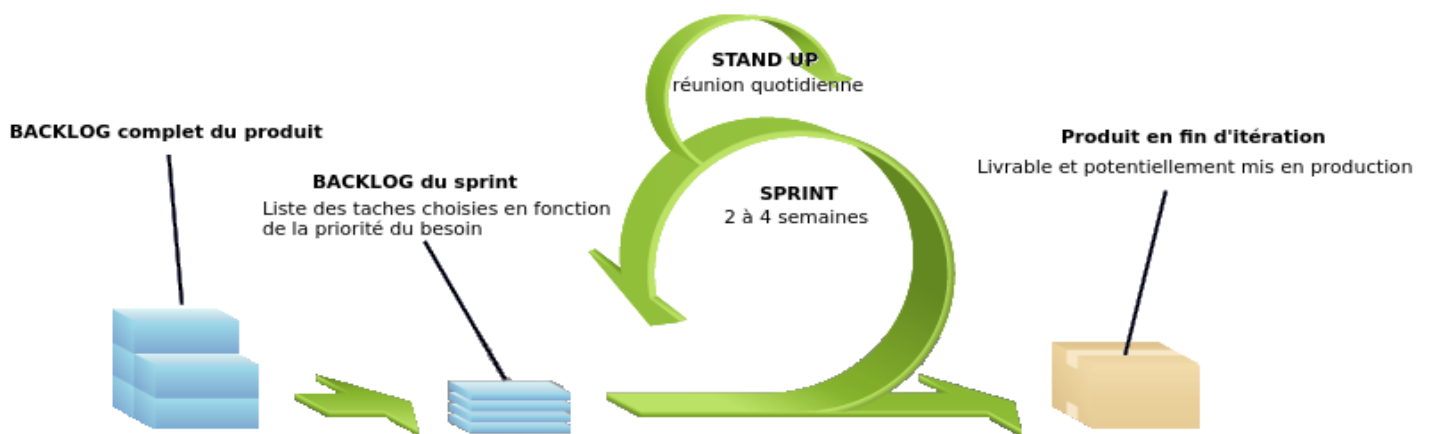


Figure 4 - Itération selon la méthodologie Agile Scrum

1.3.2 Présentation du mammographe et du logiciel

Mammographe Pristina

Pristina est un mammographe développé et commercialisé par General Electric Healthcare. Il se distingue des autres mammographes par plusieurs innovations :

- **Design et ergonomie** : le dispositif est plus agréable et moins anxiogène pour la patiente. Une molette actionnée par celle-ci permet une compression du sein plus importante que lors d'une compression effectuée par le radiologue, ce qui améliore en plus la qualité d'image. La patiente a donc une prise sur le déroulement de sa mammographie. De nombreux retours ont montré que les patientes trouvaient l'examen bien moins douloureux qu'avec d'autres dispositifs médicaux.
- **Possibilité de réaliser des images 2D et 3D** : Une partie articulée de la machine permet de prendre plusieurs images sous différents angles. Un logiciel de reconstruction rassemble les images afin de créer une vue en trois dimensions du sein. Cela permet notamment de placer précisément des points de repère afin d'effectuer une biopsie (prélèvements dans le sein).
- **Plus faible exposition aux rayons X** : La dose de rayons X délivrée par la machine est moins importante que pour d'autres dispositifs, et produit une image de haute qualité.
- **Possibilité de réaliser des angiommammographies** : Il est possible d'injecter un produit de contraste à la patiente, afin de mieux déceler des anomalies, comme des tumeurs. Dans ce cas, deux images sont prises et recombinaison, afin de n'afficher que le produit de contraste, qui met en valeur le réseau de vaisseaux sanguins.
- **Mode « routine » et « biopsie »** : Le mammographe Pristina permet deux modes d'utilisation. Le premier, la « routine », correspond aux mammographies classiques de dépistage. Pendant cet examen, si le praticien détecte une anomalie, il peut proposer à la patiente d'effectuer une biopsie du sein. Pour cela, il peut utiliser le mode « biopsie », qui permet d'effectuer des images en trois dimensions, de placer précisément des points de repère, et de prélever les échantillons voulus grâce à une aiguille placée sur la machine.



Figure 5 - Mammographe Pristina et sa station d'acquisition

L'équipe Software Apps Mammography travaille sur le logiciel de la station d'acquisition du mammographe Pristina. Elle permet de choisir le mode d'acquisition d'images (routine ou biopsie), d'effectuer les réglages à distance, de prendre les images et les afficher sur l'écran. Il est notamment possible de tracer sur l'écran des objets graphiques, comme des segments ou des ellipses, afin de souligner une zone d'intérêt, comme par exemple une tumeur, dans le cas d'une angiommammographie.

Angiomammographie (CESM)

Une angiommammographie est une mammographie effectuée après l'injection d'un produit de contraste iodé à la patiente. Lorsque le produit s'est diffusé dans les vaisseaux sanguins, la machine effectue deux images : une à haute énergie, l'autre à faible énergie. Ces deux images sont recombinaisonnées, afin de ne faire apparaître plus que le produit de contraste. Les tumeurs, en particulier, sont très vascularisées : le produit sera alors présent en majorité dans cette zone. Cette technique permet de mettre en valeur uniquement les zones présentant des anomalies, afin de mieux déceler des éventuelles tumeurs cancéreuses. (Cf. ANNEXE 1)

Le document de l'annexe 1 illustre l'exemple d'une radiographie effectuée avec et sans produit de contraste, afin de mieux comprendre et visualiser la différence. Lorsque le produit de contraste n'est pas présent, il est extrêmement difficile de remarquer la tumeur, qui est bien présente. En effet, les autres structures du sein (graisse, canaux galactophores, etc.) masquent cette zone. Grâce au produit de contraste, la tumeur est bien visible par rapport au reste de la radiographie. Cela permet au médecin de déterminer plus rapidement et facilement la position de la tumeur, afin d'effectuer par la suite une biopsie, qui révélera le caractère bénin ou malin de la tumeur.

A l'inverse, il serait possible de remarquer une zone anormale sur l'image sans produit de contraste. En cas de doute, une image CESM peut être effectuée, afin de confirmer ou infirmer les doutes du clinicien. Si la radiographie CESM ne révèle rien d'anormal, alors il est fort probable que la zone ne soit pas une tumeur.

Intérêt du calcul des valeurs statistiques

Il existe un intérêt de connaître les valeurs statistiques, notamment la moyenne et l'écart-type.

Dans le cadre d'une image CESM, deux applications du calcul de ces valeurs sont possibles.

Tout d'abord, cela permet aux médecins médicaux de contrôler la qualité du dispositif médical. En effet, il existe des tables de correspondance entre des éléments du sein (kystes, tumeurs, graisse, etc.) et la moyenne de la valeur des pixels de la zone sur l'image. En effet, l'image est obtenue grâce à l'émission de rayons X. La surface traversée par ces rayons va plus ou moins les absorber, en fonction de sa densité. Or dans le sein, chaque élément a une densité différente (peau, graisse, tumeur, etc.). Il est donc possible de savoir quel élément est traversé en fonction du taux d'absorption des rayons X qui donnera une couleur différente sur l'image.

En se servant de modèles de référence, dont on connaît précisément la nature des éléments qui les composent, ainsi que de cette table, il est possible d'évaluer la qualité du mammographe, en prenant des images à partir de ces modèles. Si la moyenne des pixels d'une zone correspond à la valeur théorique, alors le dispositif permet d'effectuer des images de qualité.

Il existe également une application clinique. En effet, l'annexe 1 montre une mammographie avec et sans produit de contraste. Afin de mieux visualiser la différence, une image ayant une tumeur visible et identifiable a été choisie. Dans la réalité, les zones possédant des anomalies potentielles ne sont pas toujours aussi nettes. Le radiologue peut donc, en cas de doute, vérifier la moyenne et l'écart-

type des pixels de la zone recherchée. Il peut ainsi vérifier la nature de l'anomalie, et ainsi choisir une action en conséquence (effectuer une biopsie, par exemple).

Cette nouvelle fonctionnalité sera donc utile et utilisée par les physiciens médicaux et les praticiens pendant l'analyse des mammographies.

Présentation du logiciel

Le logiciel sur lequel porte ce stage est un outil permettant aux utilisateurs (radiologues ou physiciens) d'afficher les images de mammographie.

L'utilisateur peut de plus modifier le contraste, le facteur d'agrandissement (zoom), et placer sur l'image des points de repère, des segments ou des ellipses, afin de mettre en évidence des zones d'intérêt sur la radiographie, comme des microcalcifications, tumeurs, etc. Une capture d'écran du logiciel est présentée en ANNEXE 2.

2. Objectifs du stage

L'objectif de ce stage se décline sous quatre axes principaux :

Tout d'abord, l'**obtention de la valeur des pixels inclus dans une ellipse**. Actuellement, l'ellipse est affichée par-dessus l'image, comme un calque que l'on poserait en superposition. Il faudra donc trouver un moyen de d'obtenir la valeur des pixels inclus dans cette ellipse.

Ensuite, les **calculs statistiques sur ces valeurs**, afin d'obtenir les valeurs minimales, maximales, la moyenne et l'écart-type.

Actuellement, seule l'aire est indiquée, car elle ne dépend que de la position de l'ellipse. De même, la longueur et l'angle du segment sont aussi affichés. Ceux-ci sont calculés et affichés uniquement au niveau de l'interface graphique, codée en Qt/QML. On appelle cette partie le « front-end » : c'est avec lui qu'interagit l'utilisateur. Cette partie gère l'affichage des images, des boutons, etc. Il faudra donc faire en sorte d'effectuer les calculs dans la partie de traitement des données, codé en C++, au lieu de les calculer dans la partie graphique.. On appelle cette partie le « back-end » : il interagit avec le front-end afin de traiter les événements de l'utilisateur et d'effectuer les calculs en conséquence.

Les calculs d'aire, de longueur et d'angle ne dépendent pas des pixels de l'image. Cependant, on les regroupe avec les calculs statistiques pour davantage de cohérence dans le code.

Enfin, l'**affichage des résultats dans le logiciel**. Les calculs doivent être faits dans la partie back-end, et affichés dans la partie front-end. Il faudra donc comprendre comment fonctionne un fichier QML, et comment modifier les fichiers existants, afin d'afficher toutes les valeurs demandées.

Une fois ce travail effectué, il faudra **effectuer des tests**, afin de vérifier la fiabilité des résultats. On vérifie d'abord qu'il n'y a pas eu de régressions par rapport à la version originale du logiciel : les calculs effectués dans la partie back-end doivent donner les mêmes résultats que ceux calculés initialement dans la partie front-end. Puis, on crée des tests unitaires (qui vérifient le bon comportement des fonctions de calcul), et des tests automatiques en situation réelle (qui vérifient l'exactitude des données). Enfin, il faudra réfléchir au moyen d'**optimiser** le code, afin de réduire au maximum le nombre de calculs effectués).

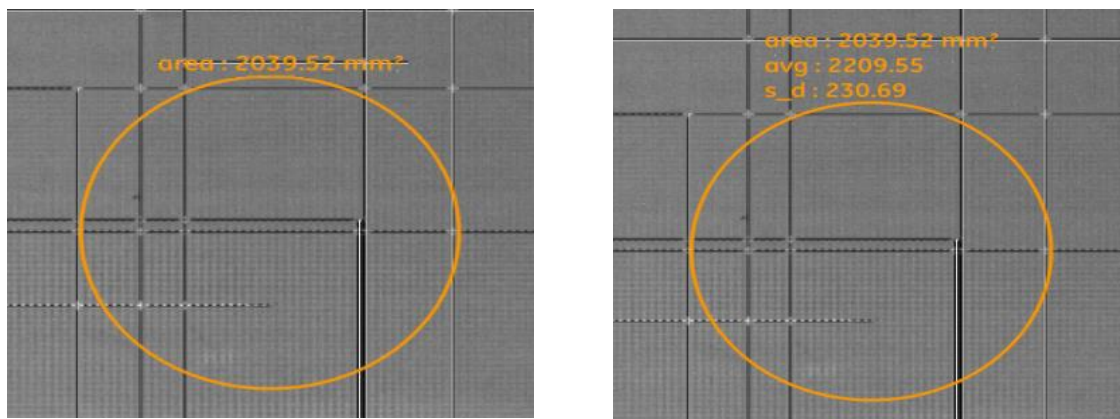


Figure 6 - Affichage d'origine (à gauche) et résultat attendu (à droite)
avg : moyenne (« average » en anglais)
s_d : écart-type (« standard deviation » en anglais)

3. Travail réalisé

3.1 Travail préliminaire

L'objectif de cette partie est d'implémenter des algorithmes de recherche du minimum, maximum, moyenne et écart-type de la valeur des pixels inclus dans une ellipse.

Dans un premier temps, il est nécessaire de procéder à une analyse théorique de la partie mathématique du sujet, et dans un second temps de l'implémenter dans le logiciel.

L'ellipse, dans notre programme, est créée à partir de deux points (xStart, yStart) et (xEnd, yEnd). Ces points définissent un rectangle englobant. A partir de ces coordonnées, il est possible de retrouver l'équation de l'ellipse.

Afin de déterminer quels pixels y sont inclus, il faut résoudre l'équation suivante:

Soit l'ellipse E de centre (a, b) , de demi rayons A et B . Soit un point $P(x, y)$ appartenant au rectangle englobant l'ellipse.

$$P(x, y) \in E \Leftrightarrow \left(\frac{x-a}{A}\right)^2 + \left(\frac{y-b}{B}\right)^2 < (1 - \varepsilon)$$

Avec :

$$a = \frac{xStart + xEnd}{2} \quad A = \frac{xEnd - xStart}{2}$$

$$b = \frac{yStart + yEnd}{2} \quad B = \frac{yEnd - yStart}{2}$$

ε représente le coefficient d'incertitude, égal à 10^{-5} .

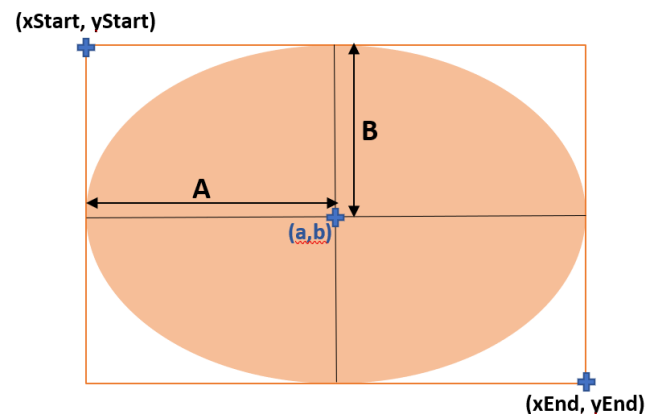


Figure 7 - Schéma d'une ellipse

Un algorithme permet de parcourir le rectangle englobant l'ellipse, et détermine quels pixels y sont inclus (ceux qui vérifient l'inéquation ci-dessus). Puis il remplit et retourne un tableau avec la valeur de ces pixels. L'ellipse est uniquement affichée pour l'utilisateur, mais la récupération des pixels ne se fait qu'à partir de ses coordonnées. On projette de façon théorique l'ellipse sur l'image : on reporte les coordonnées de l'ellipse sur l'image, et on vérifie que les coordonnées d'un pixel vérifient l'équation de cette ellipse projetée. Le schéma de cette projection est présenté en ANNEXE 3.

Ce tableau de pixels constituera les données d'entrée des algorithmes de recherche de minimum, maximum, moyenne et écart-type.

Rappel mathématique :

On considère un tableau de valeurs. L'écart-type (*standard deviation (SD) en anglais*) de ces valeurs est donné par la formule :

$$SD = \sqrt{\frac{\sum_{i=0}^n (tab[i] - average)^2}{n}}$$

Avec :

- $tab[i]$: la valeur du tableau à l'indice i
- $average$: valeur moyenne du tableau
- n : taille du tableau

L'écart-type est une mesure de dispersion autour de la moyenne. Un écart-type faible indique que toutes les valeurs sont homogènes (et donc proches de la moyenne), tandis qu'un écart-type élevé indique une grande hétérogénéité des valeurs (valeurs très dispersées autour de la moyenne).

Programme simplifié

Avant d'utiliser le code existant du logiciel, des fichiers indépendants au logiciel ont été créés, afin d'implémenter et tester les fonctions.

Initialement, une matrice représente l'image : les valeurs de la matrice représentent les valeurs des pixels. Les calculs sont faits à partir de cette matrice, pour plus de simplicité. Cette simplification se justifie par le fait qu'une image peut être considérée comme une matrice de pixels. Ainsi, il sera aisé de transposer ce programme sur le logiciel réel.

On teste tout d'abord l'algorithme de recherche de pixels inclus dans une ellipse. Afin de visualiser le bon fonctionnement de notre fonction, on initialise notre matrice à zéro. Puis on parcourt le rectangle englobant, et on modifie la valeur des cases de la matrice si et seulement si la case est incluse dans l'ellipse. Ici, on a arbitrairement fixé la valeur 10000, pour aider à mieux visualiser les résultats.

On obtient :

```
Matrice :
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 10000 10000 10000 0 0 0
0 0 0 10000 10000 10000 10000 10000 0 0
0 0 10000 10000 10000 10000 10000 10000 10000 0
0 0 10000 10000 10000 10000 10000 10000 10000 0
0 0 10000 10000 10000 10000 10000 10000 10000 0
0 0 0 10000 10000 10000 10000 10000 0 0
0 0 0 0 10000 10000 10000 0 0 0
0 0 0 0 0 0 0 0 0 0
```

Figure 8 - Premier résultat : visualisation d'une ellipse dans une matrice

Ensuite, on teste nos fonctions de calculs statistiques. On modifie la matrice pour lui donner des valeurs aléatoires (ici, entre 10 et 12), et on vérifie que les résultats de la recherche de minimum, maximum, moyenne et écart-type sont justes, par rapport au tableau de pixels que l'on récupère grâce à l'algorithme précédent. On obtient :

```
Matrice :
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 11 11 10 0 0 0
0 0 0 11 12 11 11 10 0 0
0 0 10 11 12 11 12 11 12 0
0 0 11 10 10 11 11 12 12 0
0 0 10 10 12 12 12 11 11 0
0 0 0 11 12 10 10 10 0 0
0 0 0 0 12 10 11 0 0 0
0 0 0 0 0 0 0 0 0 0

Tableau de pixels :
11 11 10 11 12 11 11 10 10 11
12 11 12 11 12 11 10 10 11 11
12 12 10 10 12 12 12 11 11 11
12 10 10 10 12 10 11 11 11 11

Valeurs statistiques :
Minimum : 10
Maximum : 12
Moyenne : 11.000000
Ecart-type : 0.771100
```

Figure 9 - vérification des valeurs statistiques

3.2 Premiers pas avec le logiciel

Architecture Modèle-Vue-Contrôleur (MVC)

Le logiciel a été développé selon l'architecture Modèle-Vue-Contrôleur. Ce modèle sépare le code en trois parties :

- **Modèle** : contient la partie du traitement des données. Dans notre cas, c'est lui qui crée un objet graphique et calcule les valeurs statistiques.
- **Vue** : contient la partie graphique du logiciel, en interaction avec l'utilisateur. Lorsque l'utilisateur clique sur un bouton, ou déplace la souris, des signaux sont émis et réceptionnés par le contrôleur.
- **Contrôleur** : assure la passerelle entre le modèle et la vue : il capte les signaux de la vue, et demande au modèle d'effectuer des actions.

Un schéma explicatif est présenté en ANNEXE 4.

Spécialisation de la classe GraphicObject en Ellipse et Segment

Dans le logiciel, les segments et ellipses sont des objets de type GraphicObject. Il a fallu spécialiser cette classe en Segment et Ellipse, afin de placer dans ces nouvelles classes les méthodes de calcul des longueurs et des valeurs statistiques.

Afin de réaliser cette première tâche, Il a été nécessaire d'utiliser les connaissances en programmation orienté objet en Java, ainsi que les connaissances du langage C, acquises lors des années d'études à l'INSA Centre-Val-De-Loire. En effet, le langage C++ est similaire au C, et l'aspect programmation orienté objet qu'il permet est similaire au Java.

A l'origine, les calculs de la longueur et l'angle du segment, ainsi que l'aire de l'ellipse, étaient effectués uniquement sur la partie graphique. Les résultats étaient affichés uniquement en tant que texte sur l'écran : la partie du traitement des données du modèle MVC n'y avait pas accès.

Afin de rester cohérent au niveau du modèle MVC, une mission de ce stage a été de déplacer les calculs effectués dans l'interface graphique vers la partie « back-end ». Ainsi, tous les calculs sont situés au même endroit : dans les classes correspondantes nouvellement créées.

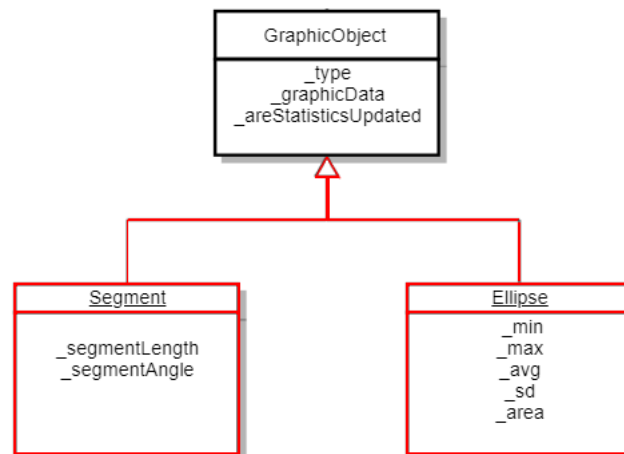


Figure 10 - Diagramme de classe - Spécialisation de la classe `GraphicObject` en `Segment` et `Ellipse`
Disponible sur l'intranet privé de General Electric, modifié le 18 Avril 2019

Obtention des pixels inclus dans une ellipse, dans le logiciel

La deuxième étape a été de tester ces nouvelles méthodes dans le logiciel, et de travailler avec l'image, plutôt qu'une matrice que l'on crée. Afin de mieux visualiser les résultats, de la même façon que précédemment, on modifie uniquement la valeur des pixels inclus dans l'ellipse. Ainsi, on vérifie que les pixels obtenus correspondent bien aux pixels de l'ellipse sélectionnée. On obtient le résultat suivant :

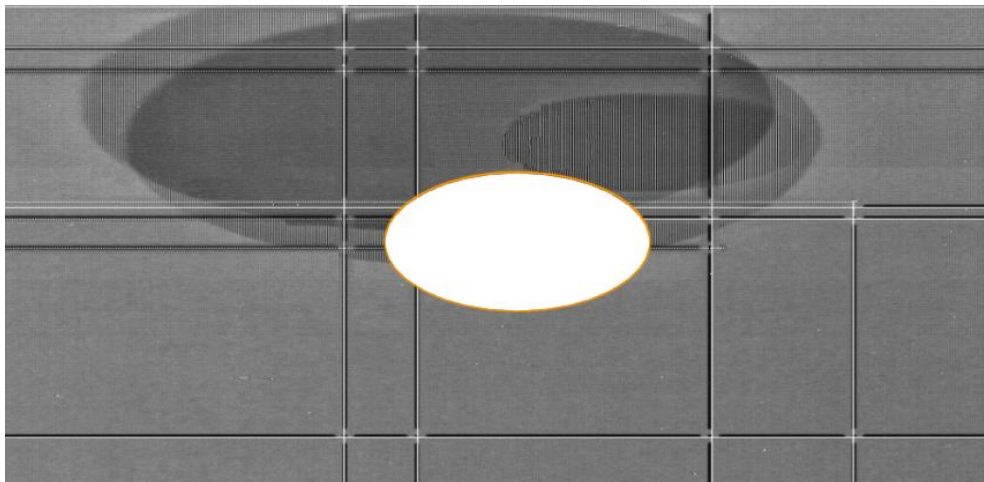


Figure 11 - Visualisation des pixels inclus dans l'ellipse (capture d'écran effectuée le 15 Avril 2019)

Difficultés rencontrées

La première difficulté rencontrée pendant ce stage a été de se plonger dans le code existant. En effet, il comporte de très nombreux fichiers, et l'architecture a été complexe à appréhender au premier abord. Il a ainsi été difficile de comprendre comment obtenir l'image, afin d'obtenir les bons pixels.

Le logiciel manipule deux versions de l'image : l'originale, et une à laquelle on applique des transformations (modifier le contraste, le zoom, ...). Dans la suite du stage, on ne travaillera qu'avec l'image originale, afin que les pixels gardent une valeur fixe, quel que soit le niveau de contraste.

3.3 Calculs statistiques lors de la création et modification d'une ellipse

3.3.1 Création d'un objet graphique

La première étape de cette partie a été de prendre le temps de bien comprendre le cheminement des données de création d'une ellipse et d'un segment, en suivant l'ordre d'appel de toutes les méthodes. Les calculs statistiques ont été mis au point d'abord lors de la création des objets graphiques (segments et ellipses), puis dans un second temps lors de la modification des objets (changement de taille ou de position sur l'image).

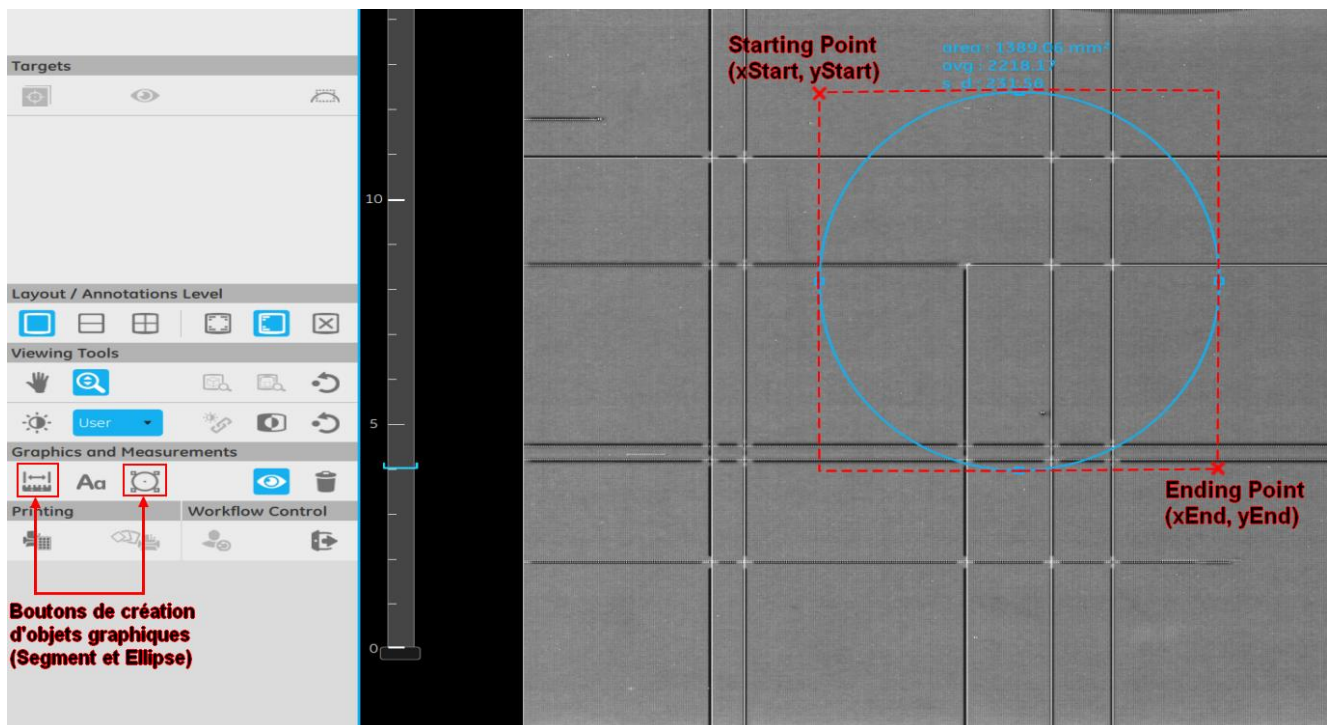


Figure 12 - Capture d'écran du logiciel lors de la création d'une ellipse. Les éléments en rouge ont été ajoutés afin d'illustrer l'image.

Le clic sur les boutons de création de segments et ellipses va provoquer l'émission de signaux. Ces signaux sont des messages transmis de la partie graphique vers la partie « contrôleur ». Ceux-ci vont être réceptionnés par le contrôleur, et des actions vont être effectuées en conséquence. Tout d'abord, le clic sur l'écran (le « Starting point ») va provoquer la création d'un objet graphique temporaire affiché à l'écran. Celui-ci va suivre la souris, afin de visualiser en temps réel sa position finale. Au deuxième clic (« Ending point »), un signal va être envoyé à la partie back-end.

Un objet graphique (Segment ou Ellipse) va être créé dans la partie back-end, afin de calculer les valeurs statistiques. Une fois l'objet créé, un signal sera envoyé vers la partie affichage du logiciel. Un objet graphique définitif va être affiché à l'écran, et l'objet temporaire sera supprimé. Les valeurs statistiques (longueur et angle pour le segment ; aire, moyenne et écart-type pour l'ellipse) sont alors affichées au niveau de l'objet graphique correspondant.

Un schéma explicatif est disponible en ANNEXE 5.

3.3.2 Modification d'un objet graphique

La modification des objets graphiques (déplacement ou modification) est similaire au chemin de la création du paragraphe précédent. Ici aussi, un signal est émis lorsque l'utilisateur clique sur l'objet graphique à modifier. Un objet graphique temporaire est affiché, afin de mieux visualiser sa nouvelle position. Lors du relâchement du clic, un signal est envoyé du front-end vers le back-end, afin de mettre à jour les valeurs statistiques. Enfin, ces valeurs sont transmises au front-end via un signal, afin de mettre les valeurs à jour à l'écran.

Afin d'augmenter la rapidité d'exécution du logiciel, il est important de réduire au maximum le nombre de calculs. Ainsi, il a fallu faire en sorte de recalculer les valeurs statistiques uniquement si

nécessaire. Le logiciel permet de scinder l'affichage en plusieurs zones, et de glisser-déposer le DataSet (i.e. l'image et tous les objets graphiques associés) dans la zone de son choix. Lors de cette opération, il n'est pas nécessaire de recalculer les valeurs statistiques, puisque ni l'image ni la position de l'objet ne sont modifiées.

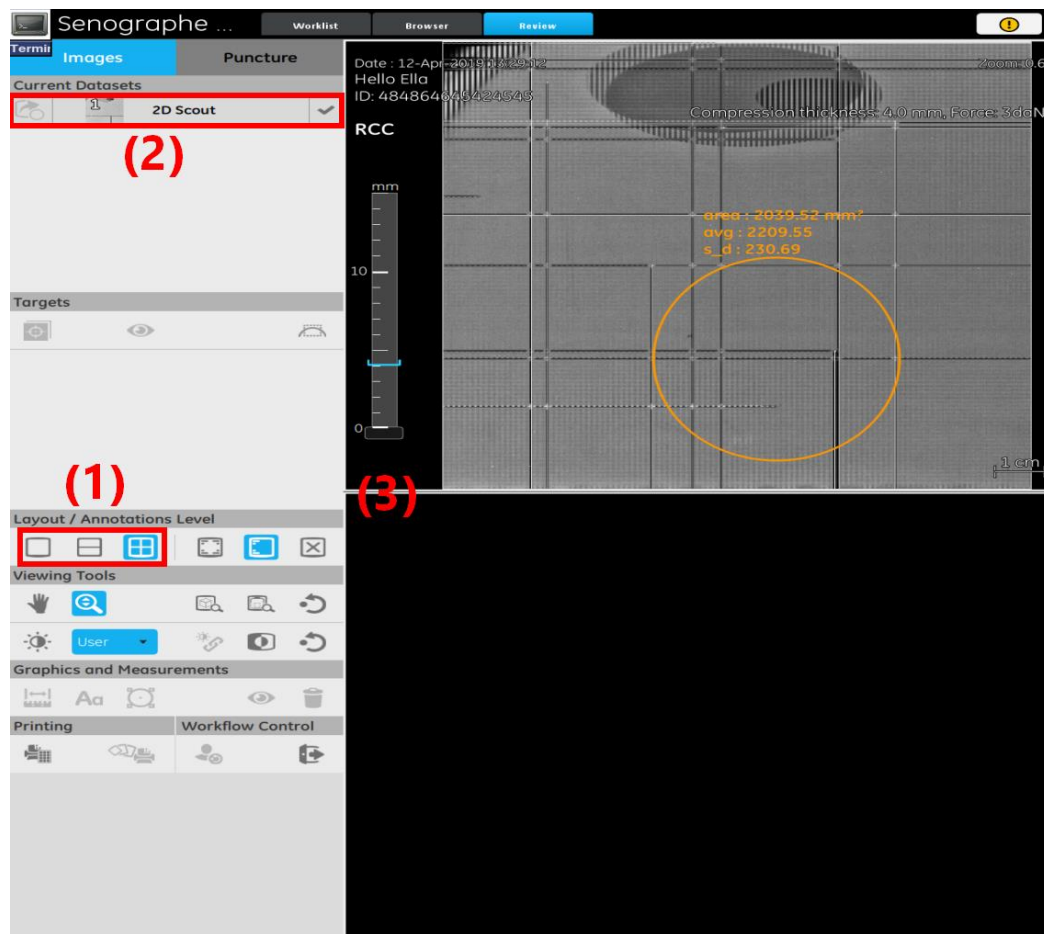


Figure 13 – illustration de l’affichage en plusieurs zones. (1) : l’utilisateur choisit le mode d’affichage, en une, deux ou quatre zones. (2) : l’utilisateur peut glisser-déposer le DataSet dans la zone (3) de son choix.

3.4 Optimisation

Le but de cette partie est d’optimiser les fonctions de calculs statistiques, afin de gagner en efficacité, en réduisant le nombre de calculs.

Obtention du tableau de pixels

Initialement, on parcourt tout le rectangle englobant l’ellipse (cf. figure 14 à gauche). Pour chaque point (x, y) dans ce rectangle, on teste s’il appartient à l’ellipse, c’est-à-dire s’il vérifie l’équation de l’ellipse. Ce test induit un nombre important de calculs pour chaque point du rectangle englobant.

Afin de réduire le nombre de calculs, on utilise les propriétés de symétrie de l’ellipse. Une ellipse possède deux axes de symétrie, représentés dans la figure suivante (axes (1) et (2)). On ne parcourt

plus tout le rectangle englobant, mais seulement un des quarts. On a choisi le quart haut gauche. Si un point A appartient à l'ellipse, alors par symétrie, les points B, C et D (respectivement les symétriques du point A par rapport à l'axe vertical, à l'origine de l'ellipse, et à l'axe horizontal) appartiennent aussi à l'ellipse. Ainsi, on pourra obtenir la totalité des pixels de l'ellipse, en ne faisant des calculs que sur un des quarts.

Cela permet de réduire par quatre le nombre d'appels à la méthode qui vérifie les coordonnées du point. On ne vérifie que pour un quart du rectangle englobant, et on en déduit les coordonnées des autres pixels par symétrie.

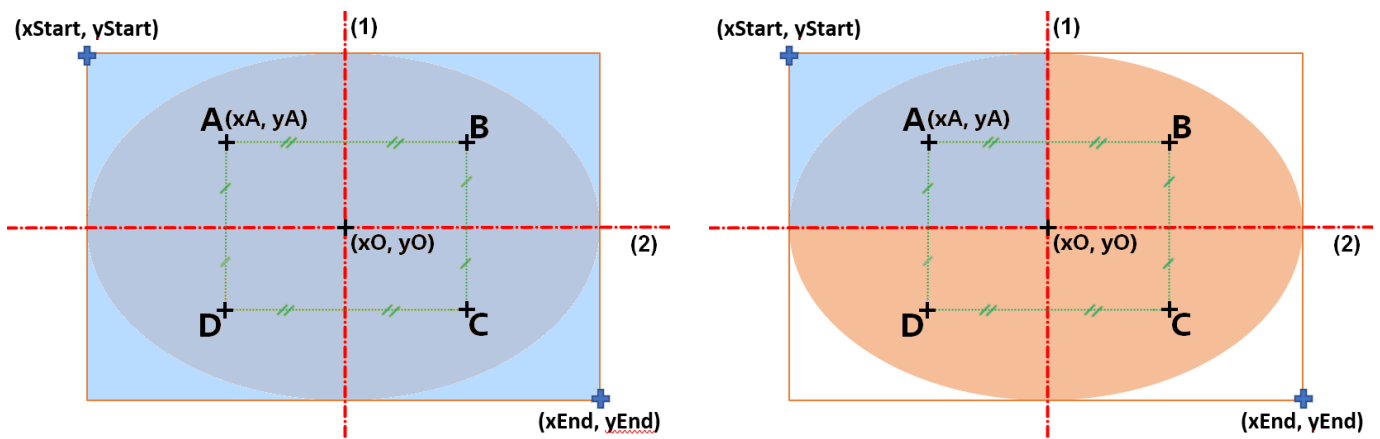


Figure 14 - Schéma illustrant l'optimisation du nombre de calculs. Les parties bleues représentent la zone parcourue par la fonction avant et après optimisation.

On aurait pu réduire encore plus le nombre de calculs, dans la méthode de recherche des pixels inclus dans l'ellipse. En effet, en parcourant ligne à ligne le rectangle englobant, lorsque l'on rencontre le premier point qui appartient à l'ellipse, alors tous les points à sa droite jusqu'au centre appartiennent à l'ellipse (cf figure 15, flèche bleue). On ne vérifiera donc les coordonnées que d'une partie du quart du rectangle englobant, sans calcul pour chaque point de l'ellipse :

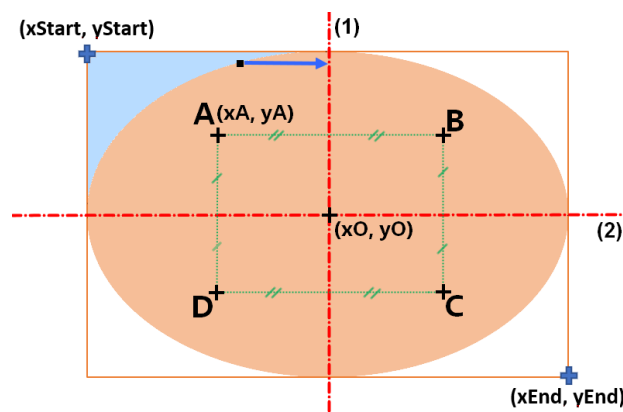


Figure 15 - Autre amélioration possible, réduisant le nombre de calculs à la seule zone bleue.

Cependant, cette méthode n'a pas été retenue car la méthode de la figure 15 rend la compréhension du code plus difficile, et le gain de performance n'est pas significatif. On se contentera donc de la solution précédente (cf figure 14, à droite)..

Parcours du tableau de pixels

Il n'est pas possible de réduire le nombre de pixels inclus dans une ellipse. En revanche, on peut faire en sorte de diminuer le nombre de parcours de ce tableau. En effet, une ellipse peut contenir plusieurs centaines de milliers de pixels (la plus grande ellipse traçable à l'écran comporte un peu plus d'un million de pixels). Il est donc important de réfléchir à des solutions pour limiter au maximum le parcours complet d'un tel tableau.

Tout d'abord, on peut optimiser l'appel des méthodes qui calculent les valeurs minimales, maximales, la moyenne et l'écart-type. Initialement, toutes ces méthodes étaient indépendantes, et appelées les unes à la suite des autres. Or dans chacune, on parcourt la totalité du tableau de pixels.

Une nouvelle méthode a donc été implémentée. Elle permet de calculer les valeurs minimales, maximales et la moyenne, en parcourant une première fois le tableau. Puis dans un second temps, on reparcourt le tableau afin de calculer l'écart-type (puisque son calcul nécessite la valeur moyenne).

Ainsi, on a divisé par deux le nombre de parcours du tableau de pixels.

3.5 Tests unitaires

Afin de vérifier le bon fonctionnement de tous les fichiers du programme, il est nécessaire d'effectuer des tests unitaires sur toutes les méthodes publiques des classes du programme.

Un test unitaire permet de vérifier le bon comportement d'une méthode. Par exemple, pour la classe `Ellipse`, des tests ont été écrits pour vérifier que les valeurs minimales, maximales, la moyenne et l'écart-type d'un tableau donné en paramètres correspondent bien aux valeurs théoriques attendues.

Il est nécessaire de créer de nombreux tests, afin de vérifier le bon comportement des méthodes, dans tous les cas de figure. Par exemple, pour la méthode qui permet de calculer le minimum des valeurs d'un tableau, on vérifie si le résultat attendu est obtenu pour un tableau de valeurs différentes de 0, un tableau ne contenant que des 0, un tableau ne contenant qu'un seul élément, un tableau vide, etc. Et on procède de même pour les autres méthodes.

Les tests du logiciel sont effectués grâce au framework **Google Test**, spécialement conçu pour tester les classes C++.

En premier lieu, il a été nécessaire d'effectuer des recherches sur l'utilisation des tests, car c'est un point qui n'a pas encore été abordé pendant le cursus suivi à l'INSA Centre-Val-De-Loire. Puis, dans un second temps, il a fallu s'inspirer des tests déjà existants dans le logiciel, afin d'en créer de nouveaux. La dernière partie a donc été l'implémentation des tests des classes `Ellipse` et `Segment`, et leur intégration dans le logiciel.

Deux nouveaux fichiers de tests ont été créés : le premier teste les méthodes de la classe `Segment`, et le second teste celles de la classe `Ellipse`. Dans chacune, on implémente des tests pour tous les cas de figure, pour toutes les méthodes. Puis un programme permet d'exécuter le fichier de test. Il permet de visualiser si l'un des tests a échoué, afin de corriger le code. Lorsque tous les tests sont fonctionnels, on obtient le résultat suivant :

```
Test project /local/data/mammoAppsLocalFiles/l-boursi/p4builds/Lucie1-BOURSIER_sarah_1_WS/build/Release/AcquisitionViewer
Start 24: AcquisitionViewer_Model_EllipseTest
1/1 Test #24: AcquisitionViewer_Model_EllipseTest ...    Passed    0.98 sec
100% tests passed, 0 tests failed out of 1
```

Figure 16 - Exemple de résultat du test de calcul des valeurs statistiques de l'ellipse

3.6 Tests automatiques SATI

Afin d'automatiser les tests, l'équipe Software Apps a développé un framework de tests automatiques, appelé SATI. Ces tests permettent de lancer le logiciel, et de simuler automatiquement des appuis de boutons, clics sur l'écran, etc. Ainsi, on peut par exemple simuler la création et la manipulation d'une ellipse ou d'un segment.

Ces tests permettent entre autres de vérifier l'exactitude des valeurs statistiques qui sont affichées au niveau des segments et des ellipses. Pour cela, on effectue l'acquisition d'une image à l'aide du mammographe Pristina. A la place du sein, on place une pièce de taille connue. Ainsi, on peut déterminer les valeurs théoriques attendues pour la longueur du segment, et pour l'aire de l'ellipse. On compare alors la valeur théorique à la valeur pratique, calculée grâce au logiciel, avec une marge d'erreur de 2%.

Le calcul de la longueur et angle du segment, et de l'aire de l'ellipse, étaient déjà effectués dans la version d'origine du logiciel. Ce test SATI a donc déjà été implémenté : il a juste fallu s'assurer qu'avec les modifications apportées, les résultats passaient le test. Cela montre qu'il n'y a pas eu de régressions entre l'ancienne version et la nouvelle version implémentée pendant le stage.

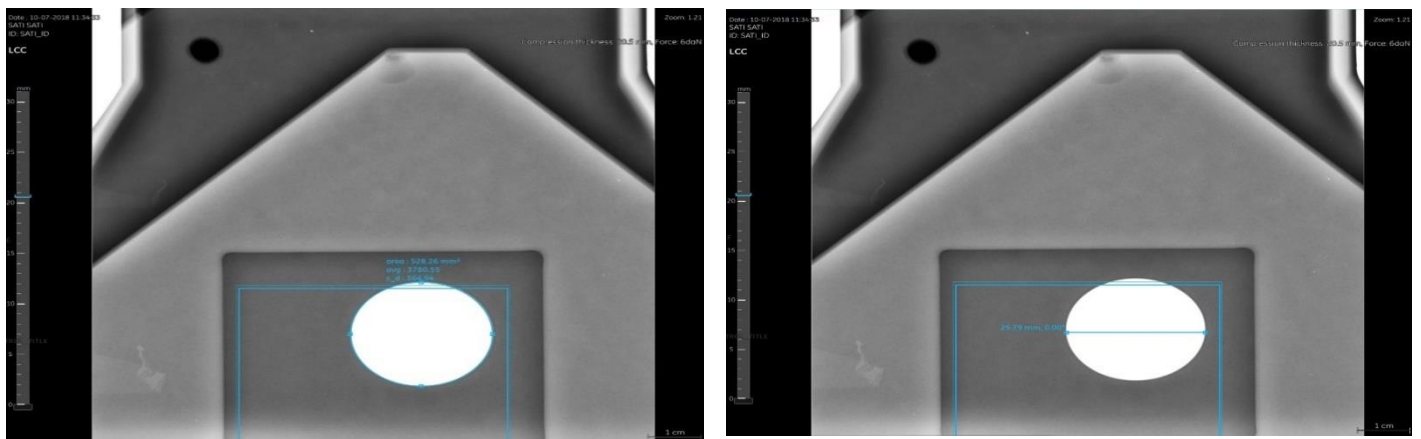


Figure 17 - Capture d'écran effectuée lors du test SATI. La partie blanche est la pièce de référence dont on connaît la taille. Ici, on teste l'aire de l'ellipse et la longueur du segment.

En revanche, le calcul des valeurs statistiques des pixels de l'ellipse a été implémenté pendant ce stage. Il a donc fallu créer un nouveau test automatique SATI afin de vérifier que les valeurs de moyenne et écart-type étaient bien conformes aux valeurs théoriques.

Dans l'image de la figure 17, on connaît la taille de la pièce en blanc, mais on ne connaît pas la valeur des pixels de l'image. Il n'était donc pas possible de l'utiliser pour tester nos valeurs statistiques. Une autre image a donc été créée. Celle-ci contient des pixels ayant une valeur aléatoire entre 0 (valeur minimale des pixels d'une image de radiographie) et 4095 (valeur maximale possible). La valeur des pixels suit une loi de probabilité uniforme.

Rappel : moyenne et écart-type d'une loi de probabilité uniforme :

- Dans une loi de probabilité uniforme, la moyenne est égale à $\frac{N+1}{2}$, avec N : le nombre de valeurs différentes. La moyenne vaut donc 2048.

- La formule de l'écart-type d'une loi uniforme est : $\sqrt{\frac{N^2-1}{12}}$, qui est égal dans notre cas à 1182.

Ici aussi, on introduit une marge d'erreur de 2% entre la valeur théorique et la valeur pratique. On obtient le résultat de l'ANNEXE 6.

On peut constater que les valeurs de moyenne et d'écart-type correspondent bien aux résultats théoriques, car ils sont inclus dans un intervalle de 2% autour de la valeur théorique. (N.B : afin de mieux visualiser les résultats, difficilement visibles dus à la couleur claire de l'ellipse, sa couleur et son contraste, ainsi que les résultats statistiques, ont été modifiés par rapport à l'affichage réel).

Ces tests ont permis de valider les résultats pratiques de calcul de moyenne et écart-type, afin d'assurer la qualité et la fiabilité des résultats du logiciel.

3.7 Corrections

L'équipe dans laquelle se déroule ce stage a fait le choix d'effectuer des revues de code régulières. Le principe est qu'avant d'ajouter du code au logiciel lui-même, il est relu par un ou plusieurs collègues. Ceux-ci peuvent laisser des commentaires, afin de proposer des éléments de correction et d'amélioration. Il y a toujours au moins une personne désignée pour valider la revue, lorsque toutes les corrections ont été effectuées. Cela permet d'ajouter une sécurité supplémentaire grâce à un regard extérieur à son travail. Cela permet, à long terme, de gagner du temps de programmation, en corrigeant son code au fur et à mesure.

Afin d'effectuer ces revues de code, on utilise le logiciel Collaborator de SmartBear Software. Ce logiciel permet de visualiser immédiatement les commentaires, le fichier d'origine, et le fichier modifié.

Ce logiciel a permis de corriger le code au fur et à mesure, et ainsi à gagner en efficacité, et en clarté.

4. Améliorations

4.1 Rotation de l'ellipse

4.1.1 Objectifs

La dernière partie de ce stage a permis de réfléchir aux possibilités d'amélioration de la solution existante. Une première piste d'amélioration est l'ajout de la possibilité d'effectuer une rotation des ellipses, grâce à un curseur que l'on déplace à la souris.

La rotation de l'ellipse est une fonctionnalité intéressante, car elle permet de mettre en valeur la zone de son choix de manière plus précise, si la zone est allongée selon un certain angle. Il a donc été décidé que la fin du stage se consacre à cet ajout.

Il faudra donc mettre à jour l'interface graphique, afin de visualiser la rotation de l'ellipse, mais également mettre à jour les méthodes de récupération des pixels inclus dans une ellipse. Actuellement, les formules ne fonctionnent que pour un angle égal à zéro. Il faudra donc mettre au point de nouvelles méthodes de calcul afin de prendre en compte un angle de rotation.

4.1.2 Démarches

4.1.2.1 Obtention des pixels dans l'ellipse

Rotation de chaque point de l'ellipse

Afin d'obtenir les pixels inclus dans une ellipse ayant un angle de rotation, il a été nécessaire de réfléchir sur papier, en posant le problème de manière théorique. Deux approches ont été abordées.

Tout d'abord, la rotation pour chaque point de l'ellipse, grâce aux formules de la matrice de rotation :

Soit α un angle de rotation en radian, et $P(x, y)$ un point de l'ellipse. Le point $P(x', y')$ résultant de la rotation de P autour du point $\Omega(a, b)$ (dans notre cas : le centre de l'ellipse) vérifie l'équation :

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x - a \\ y - b \end{pmatrix} \cdot \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix} + \begin{pmatrix} a \\ b \end{pmatrix}$$

Le problème qui s'est posé, est que les nouvelles coordonnées (x', y') ne sont pas forcément des valeurs entières. Or sur l'image, les coordonnées de pixels sont des entiers. Donc il arrivait qu'après l'arrondi en entiers, plusieurs pixels se retrouvaient au même endroit, et des espaces vides apparaissaient.

Afin de mieux visualiser les résultats, comme précédemment, on modifie la valeur des pixels de l'ellipse en blanc, après transformation. On observe les résultats de l'ANNEXE 7. On peut constater que selon l'angle choisi, les pixels forment des motifs et ne remplissent pas toute la zone. Il a donc fallu trouver d'autres méthodes afin d'obtenir un résultat plus satisfaisant.

Modification de l'équation de l'ellipse

La deuxième approche a été d'inclure l'angle de rotation dans la définition même de l'ellipse : il faut que son équation prenne maintenant en compte un angle de rotation donné. Cela éviterait de nombreux calculs de rotation de chaque point de l'ellipse, et les problèmes d'arrondis rencontrés précédemment. Pour rappel, l'équation de l'ellipse, avant la rotation, était :

$$P(x, y) \in E \Leftrightarrow \left(\frac{x - a}{A}\right)^2 + \left(\frac{y - b}{B}\right)^2 < (1 - \varepsilon)$$

En prenant en compte un angle de rotation α autour du centre de l'ellipse, comme pour le calcul précédent pour un point, l'équation devient :

$$P(x, y) \in E \Leftrightarrow \left(\frac{(x - a) * \cos(\alpha) - (y - b) * \sin(\alpha)}{A}\right)^2 + \left(\frac{(x - a) * \sin(\alpha) + (y - b) * \cos(\alpha)}{B}\right)^2 < (1 - \varepsilon)$$

Il a également été nécessaire de corriger la zone sur laquelle les calculs sont effectués. En effet, dans le cas d'un angle de rotation nul, l'ellipse possède des symétries horizontales et verticales, ce qui simplifie les calculs et permet d'effectuer des optimisations. Lorsque l'angle est quelconque, le coefficient directeur des axes de symétrie change :

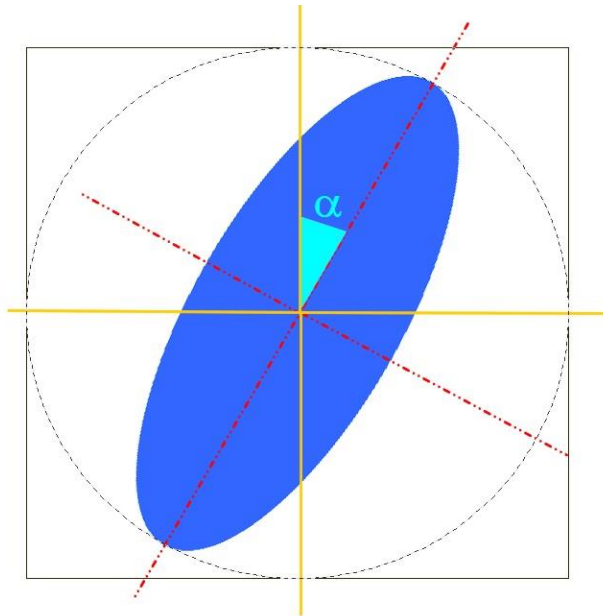


Figure 18 - Schéma représentant la rotation d'une ellipse d'un angle α par rapport à son centre

Afin d'être sûr que tous les pixels de l'ellipse sont traités, on parcourt non pas le premier quart du rectangle englobant comme précédemment, mais le carré dont le côté est le plus grand des diamètres de l'ellipse.

Cette méthode possède un avantage : tous les pixels de l'ellipse sont traités : il n'y a plus de doublons ou de zone non traitées, comme le montre l'ANNEXE 8 : on vérifie si chaque point vérifie l'équation, sans effectuer de transformation point par point. On évite ainsi les erreurs d'arrondis précédentes.

4.1.2.2 Interface graphique

Les algorithmes de récupération des pixels inclus dans l'ellipse permettent de mettre à jour les calculs statistiques, car ils ne dépendent que du tableau de valeurs de pixels obtenus. Ainsi, au niveau de la partie de traitement des données (back-end), tous les calculs prennent en compte l'angle de rotation de l'ellipse. En revanche, rien n'a encore été changé visuellement pour l'utilisateur (l'ellipse blanche que l'on affiche en annexe ne sert qu'à vérifier quels pixels sont obtenus : ils ne sont pas voués à être modifiés en blanc dans la version finale). Il a donc été nécessaire de trouver une solution graphique pour permettre à l'utilisateur d'effectuer une rotation sur l'ellipse.

L'idée est d'ajouter une ancre cliquable, comme les carrés sur le côté de l'ellipse qui modifient sa taille, et qui permettrait de tourner l'ellipse. La première approche a été de créer un point que l'on déplace selon le cercle englobant (cf. figure 18 : le cercle en pointillés), et qui permet de calculer et afficher l'angle, en fonction de la position de la souris de l'utilisateur par rapport au centre de l'ellipse.

Ensuite, cet angle est transmis à la partie back-end du programme, afin de mettre à jour l'équation de l'ellipse et les valeurs statistiques correspondantes. Le résultat est visible en ANNEXE 9.

4.1.3 Limites

L'inconvénient de l'ajout de la rotation des ellipses est que l'on augmente le nombre de calculs effectués : l'optimisation trouvée précédemment ne fonctionne plus, car elle n'est pas adaptée à des

droites de coefficient directeur quelconques. Pour rappel, l'ANNEXE 10.1 montre les zones parcourues par l'algorithme avant et après l'optimisation de la section 3.4. Du fait de l'ajout de la rotation de l'ellipse, on augmente la zone de parcours de l'algorithme en déterminant si chaque point de cette zone appartient à l'ellipse. On perd donc en efficacité, comme le montre l'ANNEXE 10.2.

La piste d'amélioration qui a été exploitée est l'utilisation de propriétés de symétrie centrale de l'ellipse. En effet, en effectuant les calculs uniquement dans une des moitiés du carré englobant l'ellipse, par symétrie centrale, il est possible de déterminer le reste des pixels (cf. points A(i, j), A'(i', j') et O(a, b) de l'ANNEXE 10.2).

En effet, en considérant, dans notre exemple, O comme milieu du segment [A, A'] et centre de l'ellipse, on a :

$$\begin{cases} a = \frac{i + i'}{2} = \frac{endX + startX}{2} \\ b = \frac{j + j'}{2} = \frac{endY + startY}{2} \end{cases}$$

On en déduit les coordonnées de A' :

$$\begin{cases} i' = endX + startX - i \\ j' = endY + startY - j \end{cases}$$

Cela permet de réduire le nombre de calculs par deux : on perd en efficacité par rapport aux optimisations précédentes (ANNEXE 10.1), mais on en gagne par rapport à la solution initiale de l'ANNEXE 10.2.

Pour le moment, même si l'on obtient les pixels de l'ellipse après rotation (correspondant aux ellipses blanches des ANNEXES 7, 8 et 9), le tracé de l'ellipse n'est pas modifié (ellipse aux contours orange), car cette modification est plus complexe, et demande plus de temps, il n'est donc pas possible de l'effectuer avant la fin de ce stage. Néanmoins, le programme et les tests (unitaires et SATI) restent valides, car on initialise l'angle à zéro à la création de l'ellipse : cela n'a donc pas d'impact sur ces derniers.

4.2 Histogrammes

Pour aller plus loin dans l'affichage des résultats statistiques, on peut imaginer un histogramme, qui représenterait le nombre de pixels en fonction de leur valeur. Etant donné que cette fonctionnalité n'est pas encore en projet, l'implémentation de cet histogramme a été faite de manière rapide et externe au logiciel.

L'idée est de récupérer la valeur des pixels de l'ellipse, à partir du logiciel. Une méthode de la classe Ellipse permet de récupérer le nombre de pixels par valeur, en remplissant un tableau à deux colonnes : dans la première, la valeur des pixels, et dans la seconde, le nombre de pixels ayant cette valeur. Puis on écrit les valeurs du tableau sur un fichier externe au logiciel.

Un programme indépendant au logiciel permet de créer un graphe à partir de ces valeurs. Le résultat est montré à la figure suivante :

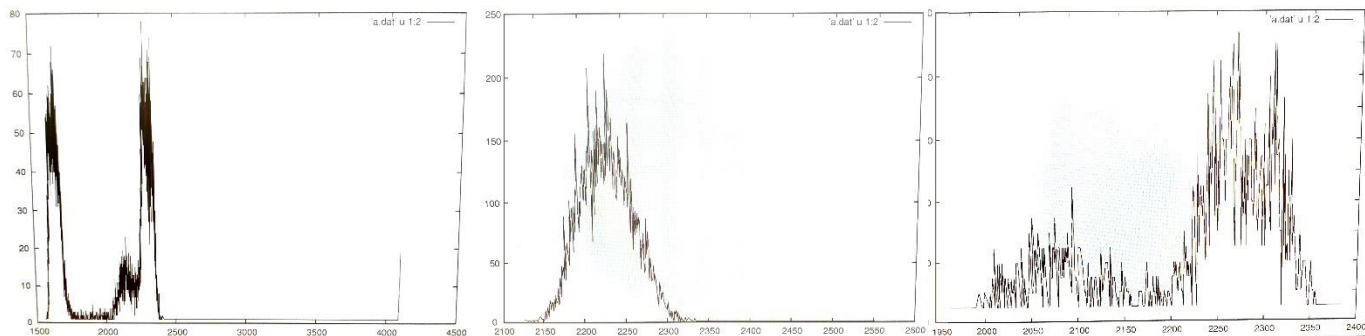


Figure 19 - Histogrammes représentant le nombre de pixels en fonction de leur valeur

Sur le premier, les deux pics signifient que deux valeurs de pixels étaient prédominantes. Dans le deuxième, les pixels sont répartis autour d'une valeur : l'écart-type était donc faible. Dans le dernier histogramme, les valeurs sont réparties sur toute la plage de valeurs : l'écart-type était donc fort, du fait de la grande hétérogénéité de valeurs.

Cette amélioration n'étant pas encore prévue dans les fonctionnalités du logiciel, on ne l'a pas implémentée entièrement dans celui-ci : il ne fonctionne qu'en externe. On pourra réutiliser ce programme si l'on souhaite ajouter cette nouvelle fonctionnalité.

Conclusion

Afin de mettre en évidence des zones d'intérêt sur une image de radiographie, les praticiens peuvent tracer des segments et ellipses à l'aide du logiciel de mammographie, développé en partie par l'équipe Software Apps de General Electric Healthcare.

Un nouveau besoin est apparu : connaître les valeurs statistiques des pixels inclus dans l'ellipse (minimum, maximum, moyenne et écart-type). Les objectifs de ce stage sont donc d'obtenir la valeur des pixels, puis de calculer et afficher les résultats des calculs statistiques.

Les objectifs ont été atteints progressivement. D'abord, de façon externe au logiciel, afin de se familiariser avec la partie mathématique (équation d'une ellipse et calculs statistiques), puis dans un second temps, avec l'implémentation de ces méthodes dans le logiciel.

Lorsque cette partie a été achevée, le sujet de stage a été enrichi. L'objectif est d'effectuer la rotation d'une ellipse, afin de gagner en précision. Il était donc nécessaire de mettre à jour les algorithmes de récupération des pixels inclus dans l'ellipse, en prenant en compte l'angle de rotation. La partie graphique a été partiellement mise à jour dans le temps restant pendant ce stage.

Un résumé de ces étapes est présenté dans le diagramme de Gantt de l'ANNEXE 11.

Ce stage m'a permis de continuer de développer les connaissances acquises pendant mon cursus à l'INSA Centre-Val-De-Loire, dans le département Sécurité et Technologies Informatiques. En particulier, il m'a permis de mettre en application mes connaissances en langage C et Java, afin d'apprendre la programmation orientée objet en C++. J'ai également acquis des connaissances en programmation avec Qt/QML, pour la partie graphique.

Travailler en entreprise m'a également appris à travailler au sein d'une équipe. J'ai également découvert la méthodologie Agile Scrum, qui permet de développer des logiciels de manière efficace. Travailler en équipe m'a également appris à respecter des règles de codage (dans un souci d'uniformité du code) ce qui m'a permis de progresser en terme de rigueur dans la programmation.

Sources

Informations générales sur General Electric

http://www3.gehealthcare.fr/fr-fr/a_propos_de_ge_healthcare , informations générales du site officiel de GE Healthcare, dernière mise à jour le 18/03/2019, consulté le 04/06/2019.

https://fr.wikipedia.org/wiki/General_Electric , informations sur le groupe General Electric, consulté le 04/06/2019

https://en.wikipedia.org/wiki/GE_Healthcare , informations sur la filiale GE Healthcare, consulté le 04/06/2019

<https://www.abcbourse.com/analyses/chiffres.aspx?s=GNep> , quelques chiffres de la société General Electric, consulté le 28/06/19

<https://www.rtl.fr/actu/debats-societe/mammographie-un-nouvel-appareil-pour-un-examen-moins-douloureux-7785072519> , avis sur le mammographe Pristina, consulté le 24/07/19

Les chiffres du cancer du sein

<https://www.e-cancer.fr/Professionnels-de-sante/Les-chiffres-du-cancer-en-France/Epidemiologie-des-cancers/Les-cancers-les-plus-frequents/Cancer-du-sein>, site officiel de l'institut national du cancer, consulté le 05/06/2019

Méthodologie Agile Scrum

https://ineumann.developpez.com/tutoriels/alm/agile_scrum/ , explications du site developpez.com, 17/06/2019

<https://www.thierry-pigot.fr/scrum-en-moins-de-10-minutes/> , définition de la méthode Agile Scrum de Thierry Pigot, 17/06/2019

<https://www.nutcache.com/fr/blog/quest-ce-quun-backlog-scrum/> , définition et explications sur le product backlog, 17/06/2019

Table des illustrations

Figure 1 – Thomas Edison (vers 1920) et Elihu Thomson (vers 1880)	5
<i>sources : https://pocketmags.com/history-revealed-magazine/july-2016/articles/25279/history-makers-thomas-edison (gauche), https://www.ge.com/reports/ge-alstom-continue-shared-history-acquisition/elihu-thomson-1880s/ (droite)</i>	
Figure 2 – Secteurs d'activités de General Electric en France	5
<i>Dernière mise à jour le 18 mars 2019 source : https://www.ge.com/fr/company/ge-en-france</i>	
Figure 3 – Organigramme simplifié du service Software Apps Mammography de General Electric Healthcare (Buc)	7
<i>source : intranet privé de GE Healthcare</i>	
Figure 4 - Itération selon la méthodologie Agile Scrum	8
<i>Image publiée le 6 décembre 2012 source : https://ineumann.developpez.com/tutoriels/alm/agile_scrum/</i>	
Figure 5 - Mammographe Prestina et sa station d'acquisition	9
<i>source : https://www.clinicalimagingystems.com/product/ge-senographe-prestina-3d-mammography-system/</i>	
Figure 6 - Affichage d'origine (à gauche) et résultat attendu (à droite) avg : moyenne (« average » en anglais) s_d : écart-type (« standard deviation » en anglais).....	12
Figure 7 - Schéma d'une ellipse	13
Figure 8 - Premier résultat : visualisation d'une ellipse dans une matrice	14
<i>Effectué le 10 Avril 2019</i>	
Figure 9 - vérification des valeurs statistiques.....	15
<i>Effectué le 10 Avril 2019</i>	
Figure 10 - Diagramme de classe - Spécialisation de la classe GraphicObject en Segment et Ellipse Disponible sur l'intranet privé de General Electric, modifié le 18 Avril 2019	16
<i>Disponible sur l'intranet privé de General Electric, modifié le 18 Avril 2019</i>	
Figure 11 - Visualisation des pixels inclus dans l'ellipse (capture d'écran effectuée le 15 Avril 2019)	17
<i>Capture d'écran effectuée le 15 Avril 2019</i>	
Figure 12 - Capture d'écran du logiciel lors de la création d'une ellipse. Les éléments en rouge ont été ajoutés afin d'illustrer l'image.....	18
<i>Capture d'écran effectuée le 14 Juin 2019</i>	
Figure 13 – illustration de l'affichage en plusieurs zones. (1) : l'utilisateur choisit le mode d'affichage, en une, deux ou quatre zones. (2) : l'utilisateur peut glisser-déposer le DataSet dans la zone (3) de son choix.	19
<i>Capture d'écran effectuée le 14 Juin 2019</i>	
Figure 14 - Schéma illustrant l'optimisation du nombre de calculs. Les parties bleues représentent la zone parcourue par la fonction avant et après optimisation.	20
Figure 15 - Autre amélioration possible, réduisant le nombre de calculs à la seule zone bleue.....	20
Figure 16 - Exemple de résultat du test de calcul des valeurs statistiques de l'ellipse	21
Figure 17 - Capture d'écran effectuée lors du test SATI. La partie blanche est la pièce de référence dont on connaît la taille. Ici, on teste l'aire de l'ellipse et la longueur du segment.	22
Figure 18 - Schéma représentant la rotation d'une ellipse d'un angle α par rapport à son centre.....	25
Figure 19 - Histogrammes représentant le nombre de pixels en fonction de leur valeur	27
<i>Histogrammes effectués le 31 Mai 2019</i>	

Glossaire

Vocabulaire de programmation

C++ : langage de programmation principal du logiciel. Créé dans les années 80, il est encore très utilisé dans les applications où la performance est importante.

QML : langage déclaratif centré sur l'interface utilisateur. Il est utilisé en complément du C++, afin de réaliser l'interface graphique avec laquelle interagit l'utilisateur. C++ et QML peuvent se communiquer des informations à l'aide de signaux.

Classe : permet de regrouper dans une même entité des données et des fonctions membres (appelées aussi méthodes) permettant de manipuler ces données. La classe est la notion de base de la programmation orientée objet. **Méthode** : partie de code qui est appelée par un nom qui est associé à un objet. Dans la plupart des cas, elle est identique à une fonction. Une méthode est toujours associée à un objet.

Objet : Élément d'une classe. On parle d'**instance** de classe.

Vocabulaire relatif au logiciel

Back-End : Partie du logiciel qui n'est pas en relation directe avec l'utilisateur, mais qui gère les événements (clic sur un bouton, mouvement de la souris, ...), les données, etc. et communique avec le front-end pour les afficher à l'écran.

Front-End : Interface graphique du logiciel, en interaction avec l'utilisateur. Il peut transmettre des données au back-end

Vocabulaire de la méthode Agile Scrum

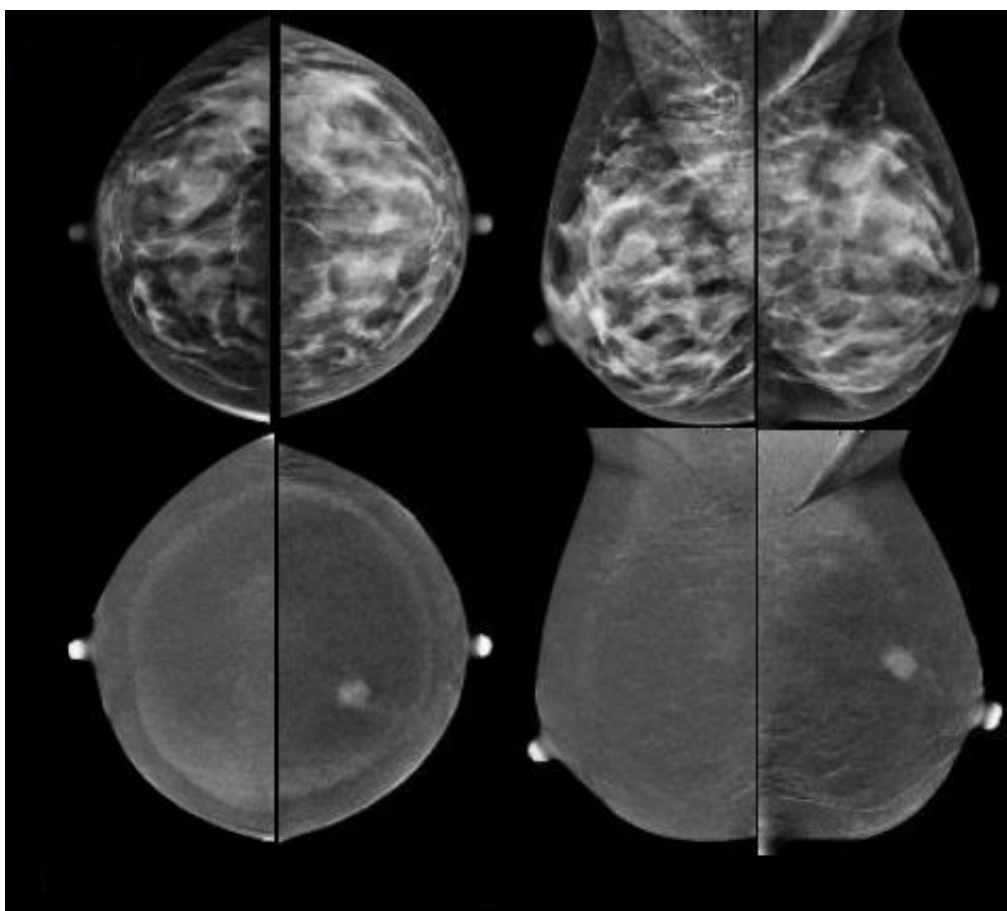
Product Backlog : destiné à recueillir tous les besoins du client que l'équipe projet doit réaliser. Il contient donc la liste des fonctionnalités intervenant dans la constitution d'un produit

Sprint : période pendant laquelle un travail spécifique doit être mené à bien avant de faire l'objet d'une révision

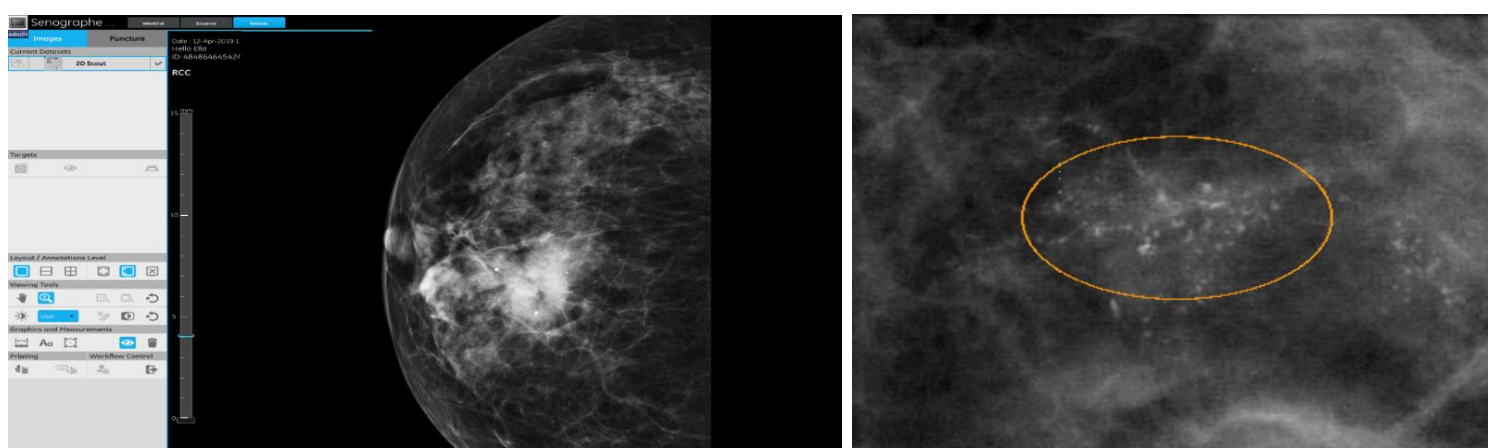
Stand-up : réunion quotidienne avec l'équipe Scrum, servant à faire le point sur les actions réalisées la veille, et qui vont être réalisées pendant la journée.

User Story : phrase simple permettant de décrire avec suffisamment de précision le contenu d'une fonctionnalité à développer.

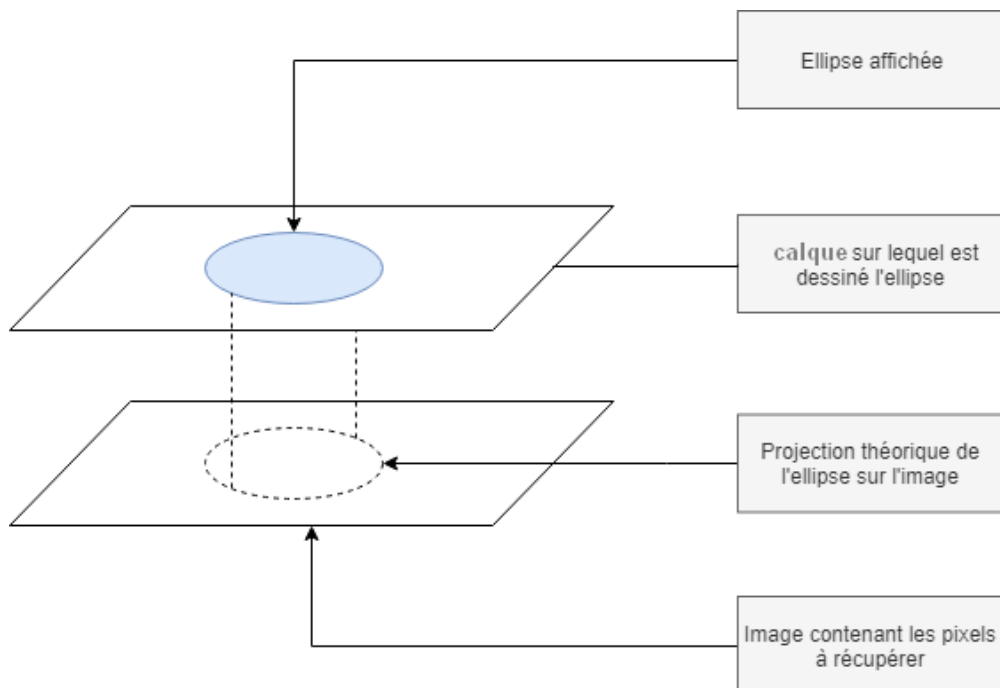
ANNEXES



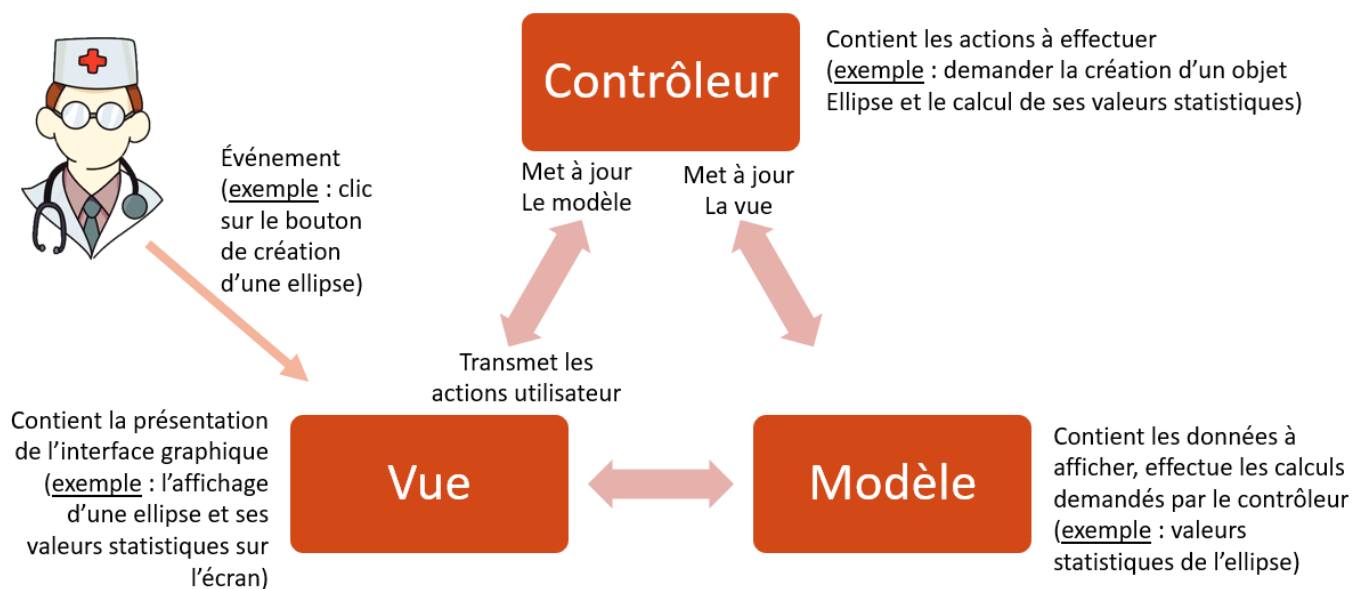
Annexe 1 - Image de mammographie sans produit de contraste (en haut) et avec produit de contraste (en bas)
Source : intranet privé de GE Healthcare, consulté le 27/06/19



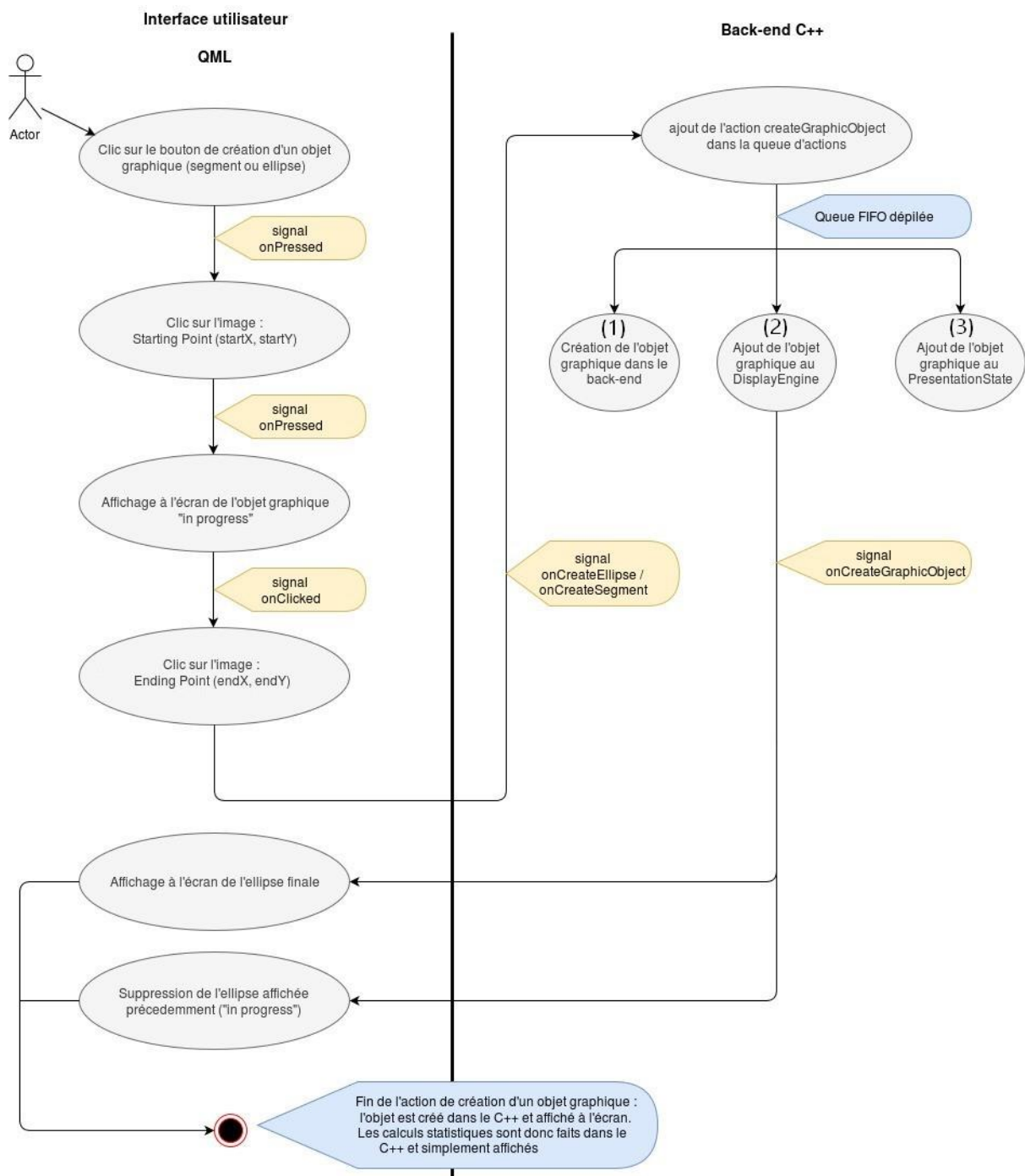
Annexe 2 – Gauche : Capture d'écran du logiciel. Les options de modification du contraste, zoom, etc.
Droite : mise en évidence d'une zone de calcifications anormales



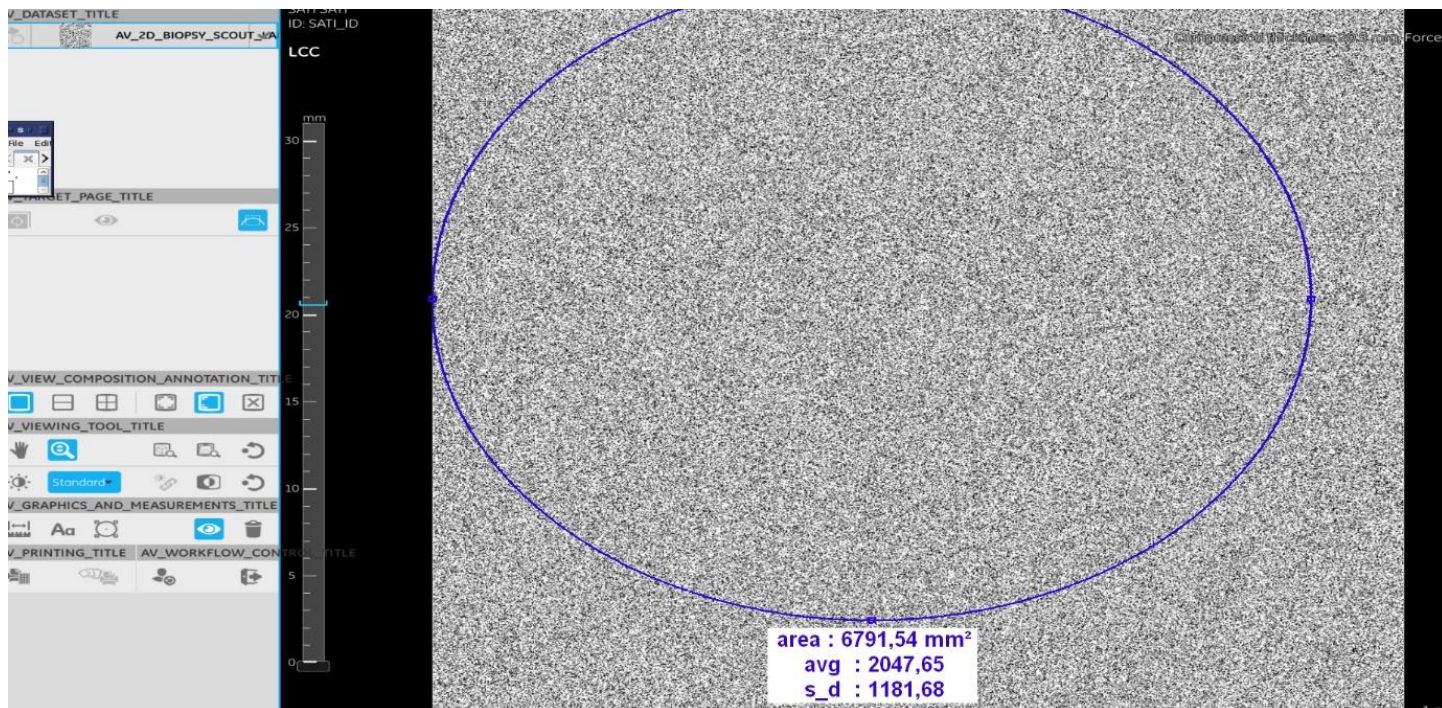
Annexe 3 - Illustration de la projection théorique de l'ellipse sur l'image, afin d'obtenir les bons pixels



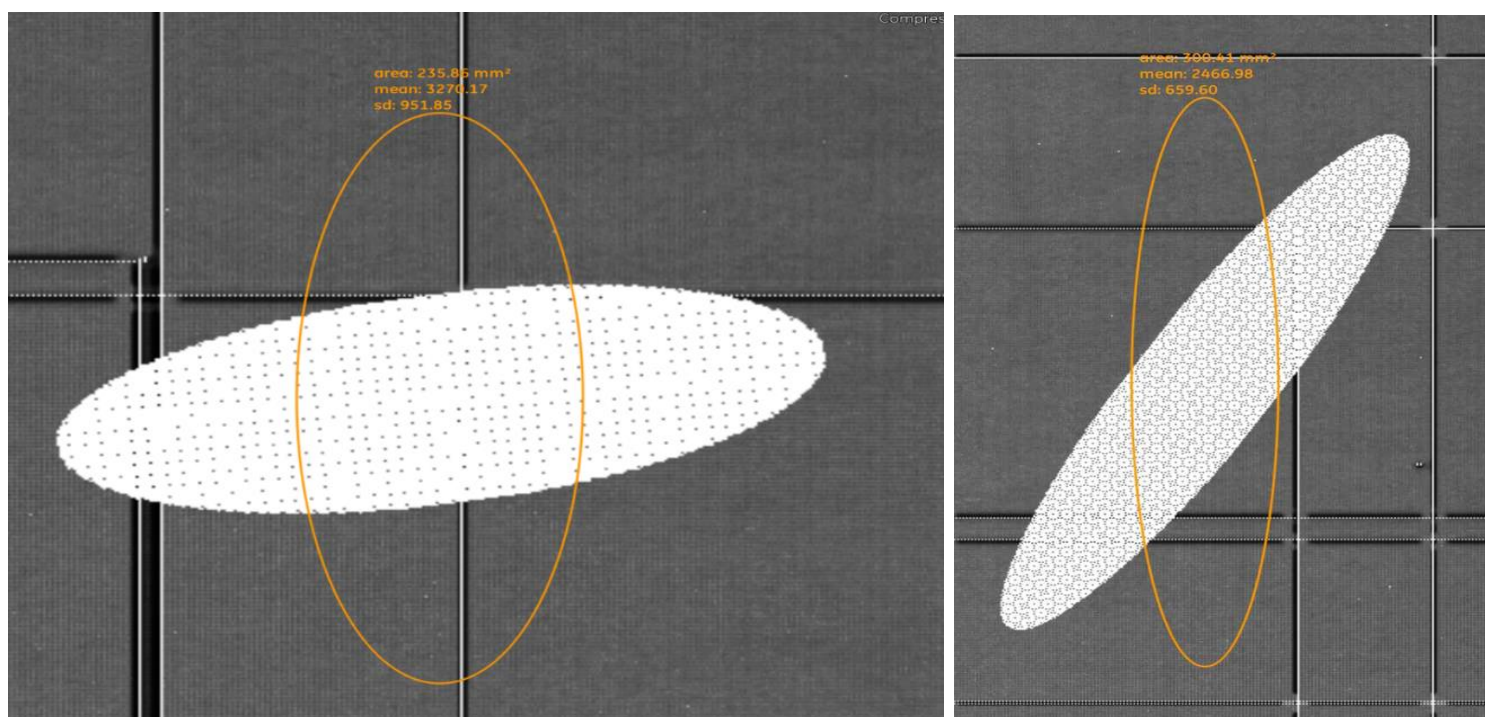
Annexe 4 - Schéma du modèle MVC, avec l'exemple de la création d'une ellipse



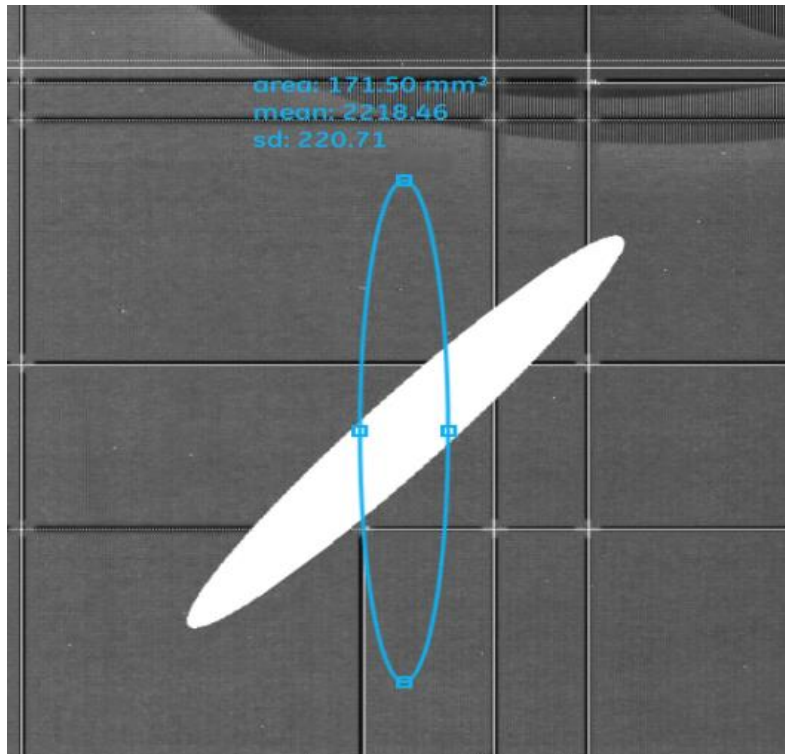
Annexe 5 - Schéma représentant le trajet des informations entre les parties QML et C++



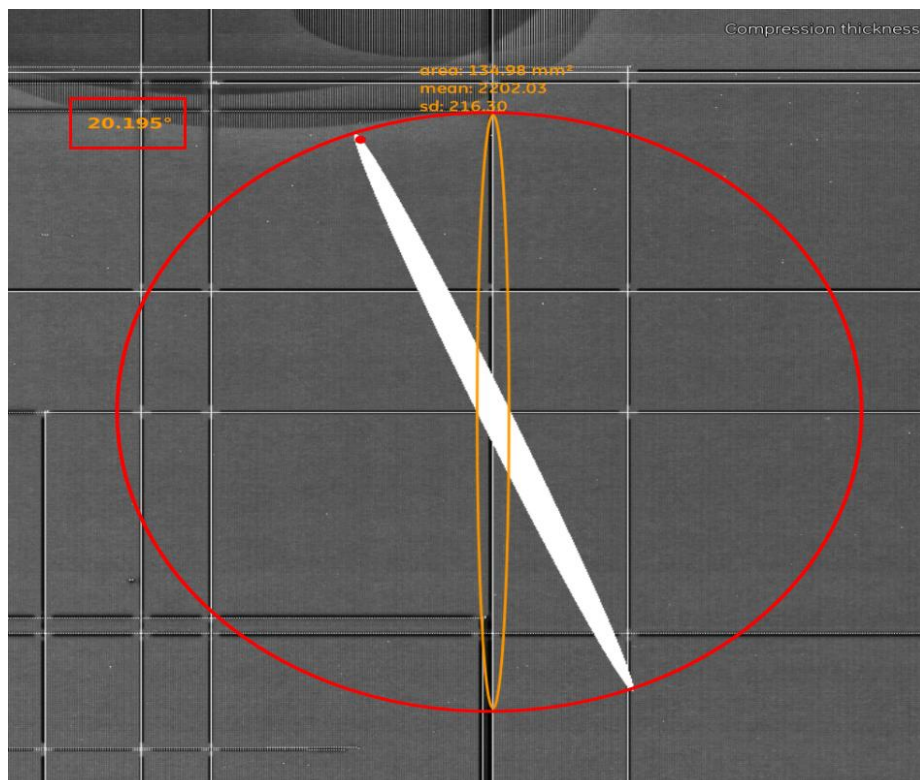
Annexe 6 - Vérification des résultats statistiques grâce au nouveau test SATI



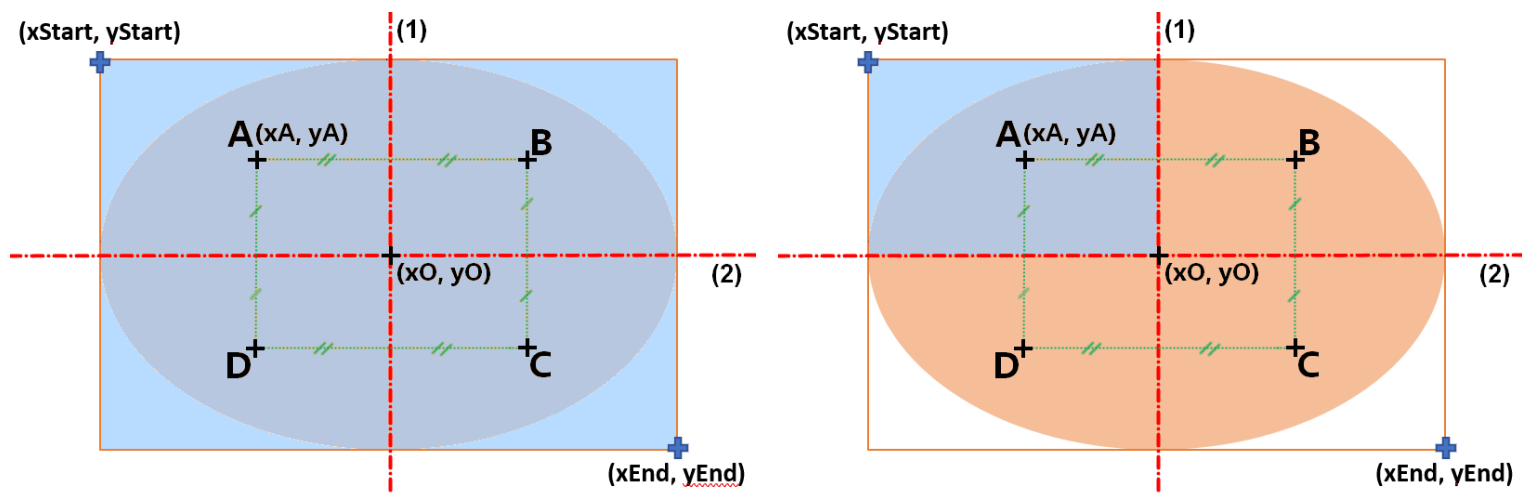
Annexe 7 - Résultats obtenus grâce à la méthode de translation point à point



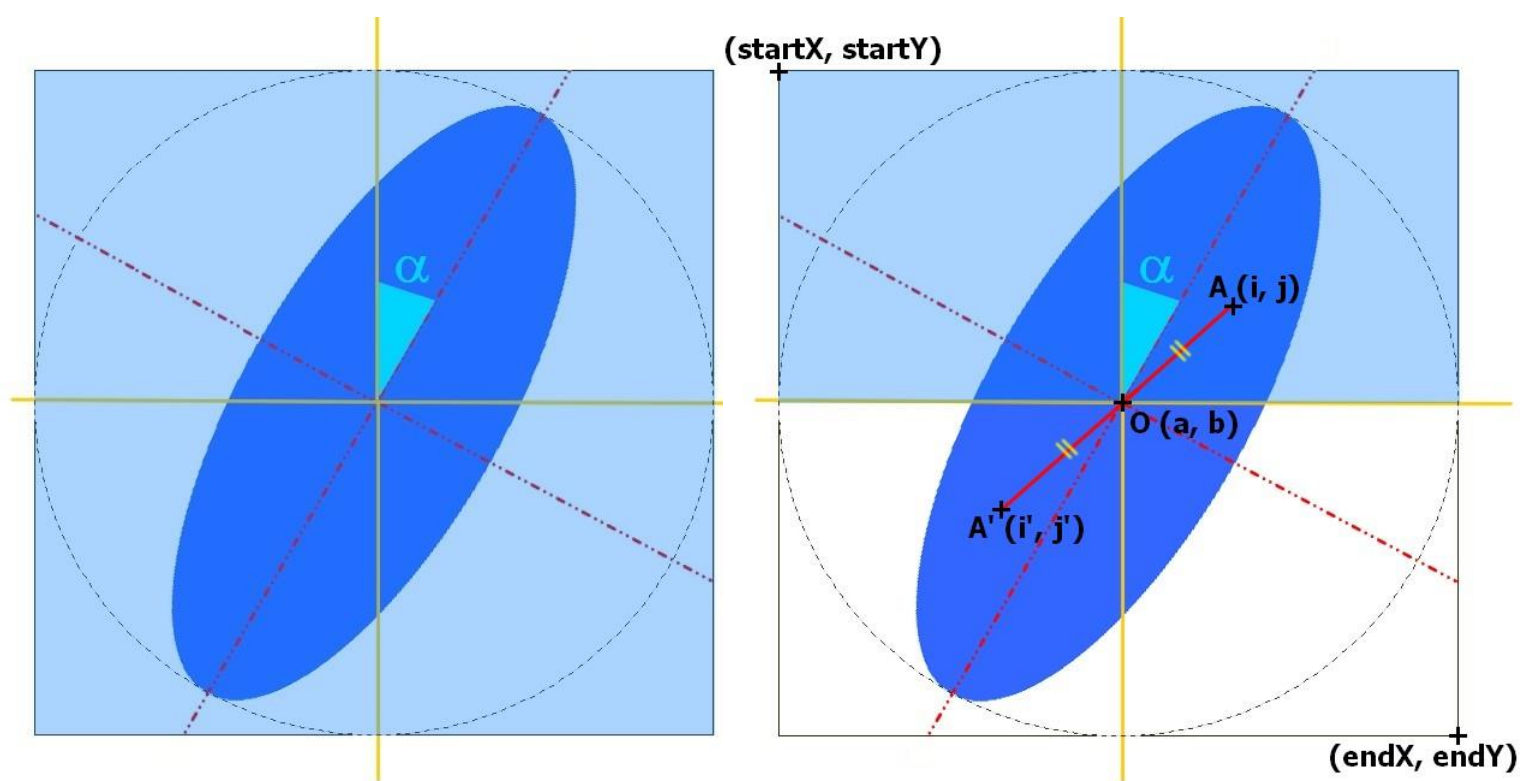
Annexe 8 - Rotation de l'ellipse grâce au changement d'équation mathématique. La zone blanche représente les pixels que l'on récupère, pour un angle donné



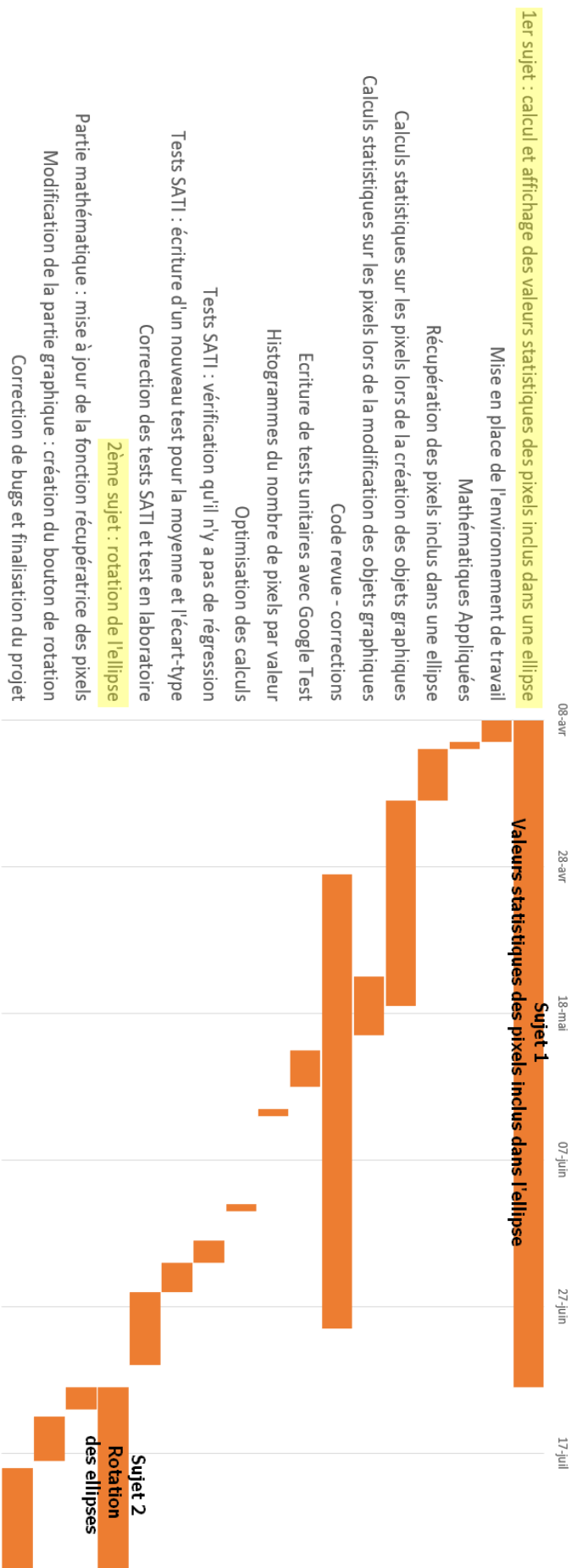
Annexe 9 - Le déplacement du point rouge permet le calcul de l'angle (haut gauche dans l'encadré). Le point se déplace selon le cercle rouge. L'ellipse blanche est ensuite tracée, afin de vérifier le bon comportement de nos méthodes.



Annexe 10.1 - Zones parcourues par l'algorithme avant l'ajout de la rotation. Gauche : zone parcourue initialement. Droite : zone parcourue après optimisation.



Annexe 10.2 - Zones parcourues par l'algorithme après l'ajout de la rotation. Gauche : zone parcourue initialement. Droite : zone parcourue après optimisation.



Annexe 11 - Diagramme de Gantt des activités réalisées pendant ce stage